

Estructuras de Datos

Clase 8 – Lista doblemente enlazada e Iteradores



Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>

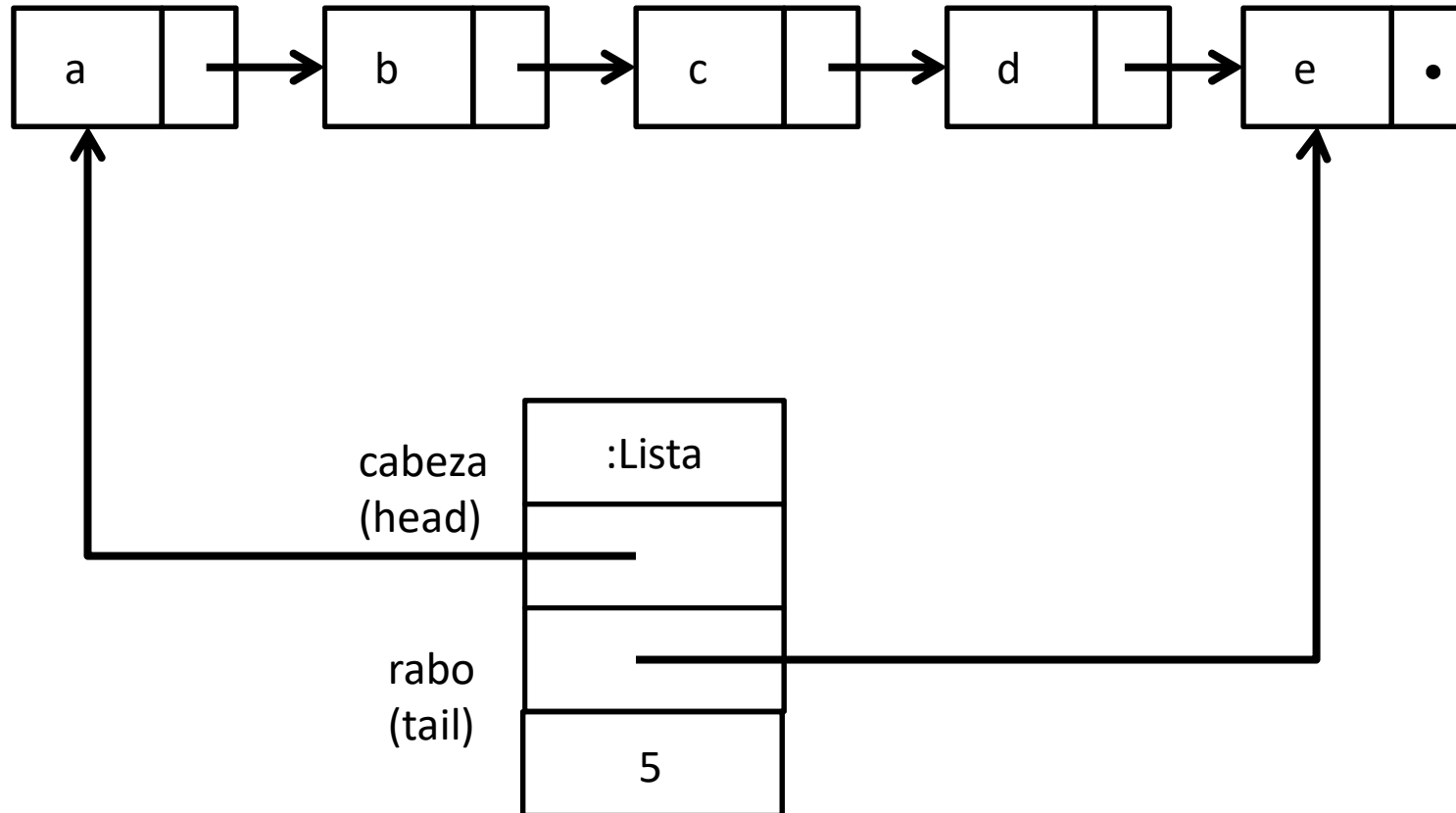


Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Temario de la clase

1. Listas simplemente enlazadas con enlace al primer y último elemento
2. Listas doblemente enlazadas con celdas centinelas inicial y final
3. Iteradores
4. Adaptadores para implementar pilas y colas con listas

Lista simplemente enlazada con enlace al principio y al final: Intuición



La referencia al último elemento permite computar `last()` y `addLast(e)` no requiere recorrido. También seguimos usando un campo para mantener la longitud y facilitar el cómputo de `size()`

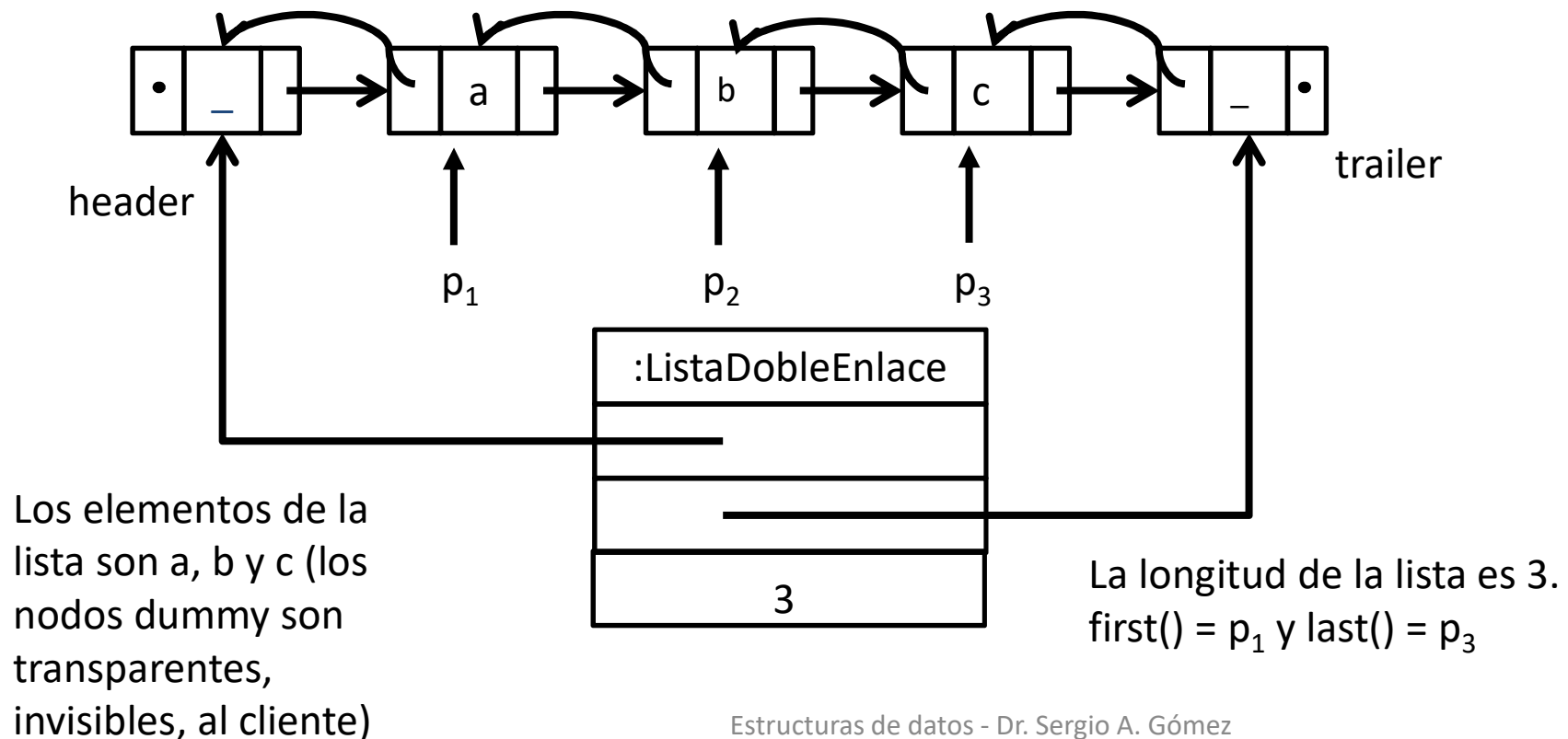
Listas con referencia al primer y último nodo:

Ventajas y desventajas

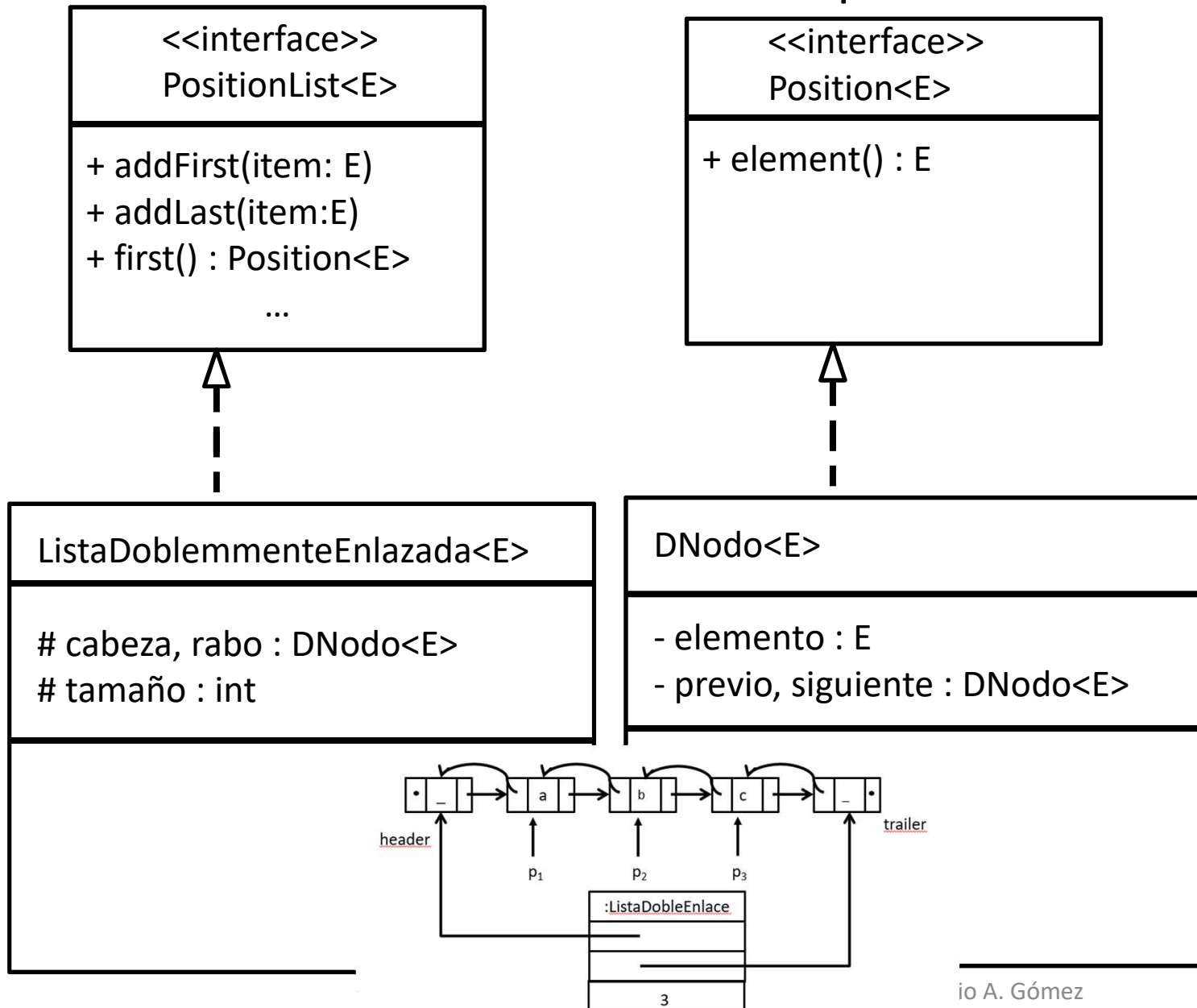
- Estructura de datos: Igual a la cola con enlaces:
 - Lista simplemente enlazada
 - Se mantiene una referencia al primer y último elemento
- Ventajas:
 - *addLast(e)* y *last()* no requieren recorrer.
- Desventajas:
 - Hay casos especiales cuando se elimina al principio y al final, cuando la lista mide 1 (igual que con la implementación de la cola enlazada).
 - *prev(p)* y *addBefore(p,x)* siguen necesitando recorrer desde el comienzo.
 - *remove(p)* y *addAfter(p,x)* requieren un caso más cuando $p = last()$ ya que requieren actualizar el rabo.

Lista doblemente enlazada: Intuición

- Cada nodo conoce un elemento, el nodo anterior y el nodo siguiente.
- La lista tiene dos nodos ficticios (*dummy*) llamados *celda de encabezamiento*, *header* y *trailer*, que evitan casos especiales al insertar y eliminar.
- La posición es directa, se conoce el primer y último nodo dummy.



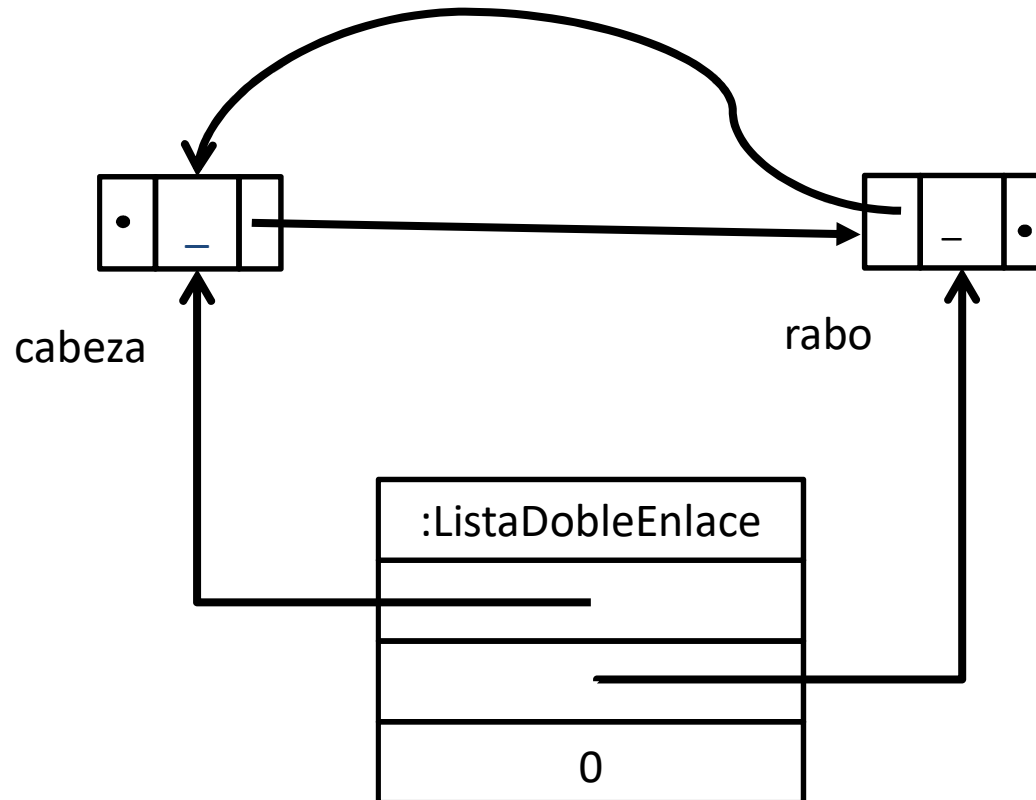
Lista doblemente enlazada: Diseño UML de la implementación



Listas doblemente enlazadas: Ventajas y desventajas

- Lista doblemente enlazada con referencia al primer y último nodo y dos celdas de encabezamiento (una al principio y otra al final).
- Ventajas:
 - Cada nodo conoce el siguiente nodo y al nodo anterior
 - No hay que recorrer para calcular el nodo anterior.
 - Al usar celdas de encabezamiento las operaciones no tienen casos especiales (e.g., casos con referencias nulas) y no hay que recorrer para computar `last()` ni `addLast(x)`
- Desventajas:
 - Mayor uso de espacio porque para una lista que tiene n elementos hay $n+2$ celdas y cada celda tiene un enlace más.
- Leer: secciones 3.3 y 6.2.4 de Goodrich & Tamassia. Allí la lista está programada casi completamente.

Lista doblemente enlazada: Constructor



La longitud de la lista es 0.
first() = error y last() = error

Lista doblemente enlazada: Constructor

```
public ListaDoblementeEnlazada() {  
    cabeza = new DNodeo<E>(null);    // El rótulo de la cabeza es null.  
    rabo = new DNodeo<E>(null);      // El rótulo del rabo es null.  
    cabeza.setPrev( null );           // La cabeza no tiene previo.  
    cabeza.setNext( rabo );           // El siguiente de la cabeza es el rabo.  
    rabo.setPrev( cabeza );           // El previo del rabo es la cabeza.  
    rabo.setNext( null );             // El rabo no tiene siguiente.  
    tamaño = 0;                      // La lista no tiene elementos.  
}
```

Lista doblemente enlazada: Traza del Constructor

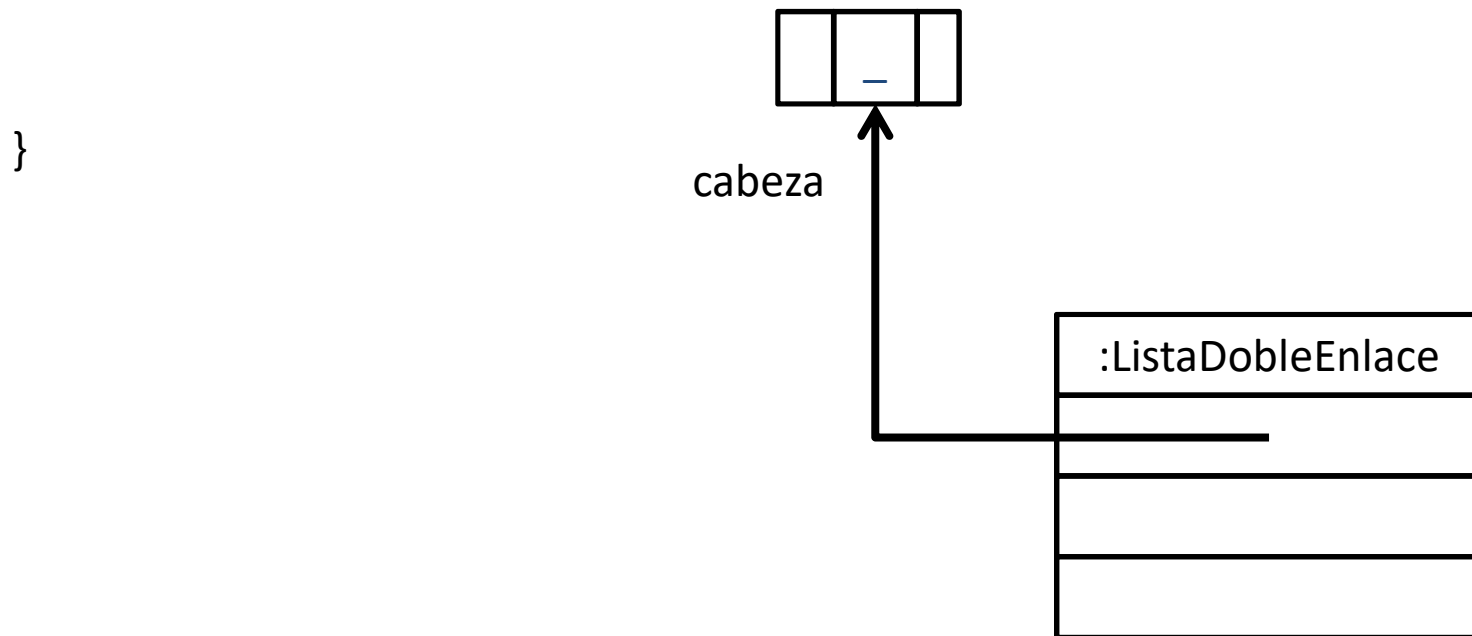
```
public ListaDoblementeEnlazada() {
```

```
}
```

:ListaDobleEnlace

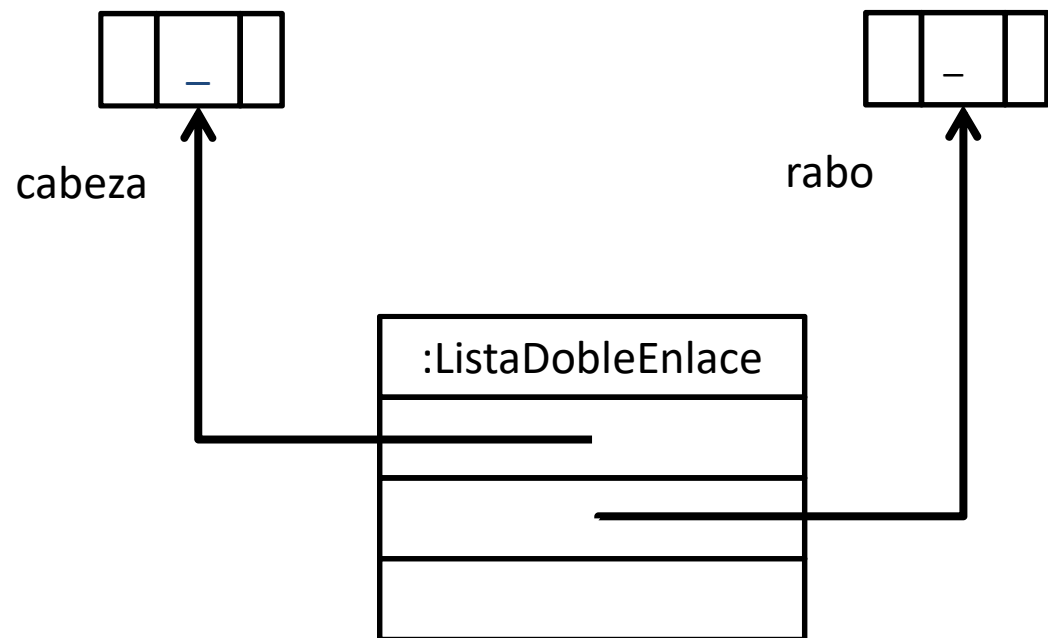
Lista doblemente enlazada: Traza del Constructor

```
public ListaDoblementeEnlazada() {  
    1. cabeza = new DNodeo<E>(null);  
}
```



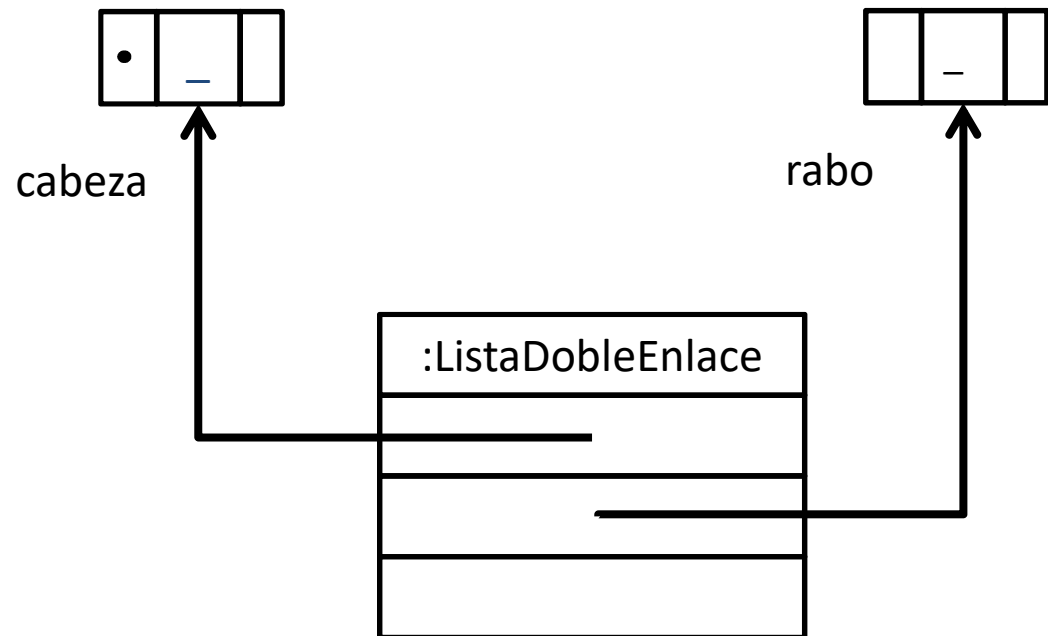
Lista doblemente enlazada: Traza del Constructor

```
public ListaDoblementeEnlazada() {  
    1. cabeza = new DNodeo<E>(null);  
    2. rabo = new DNodeo<E>(null);  
}
```



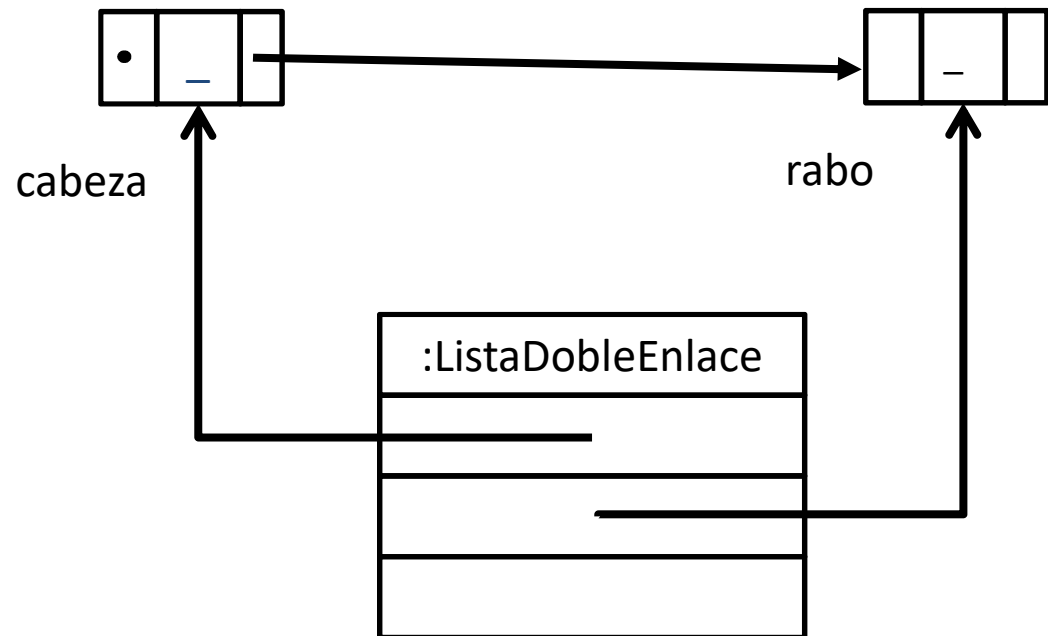
Lista doblemente enlazada: Trazo del Constructor

```
public ListaDoblementeEnlazada() {  
    1. cabeza = new DNodeo<E>(null);  
    2. rabo = new DNodeo<E>(null);  
    3. cabeza.setPrev( null );  
}
```



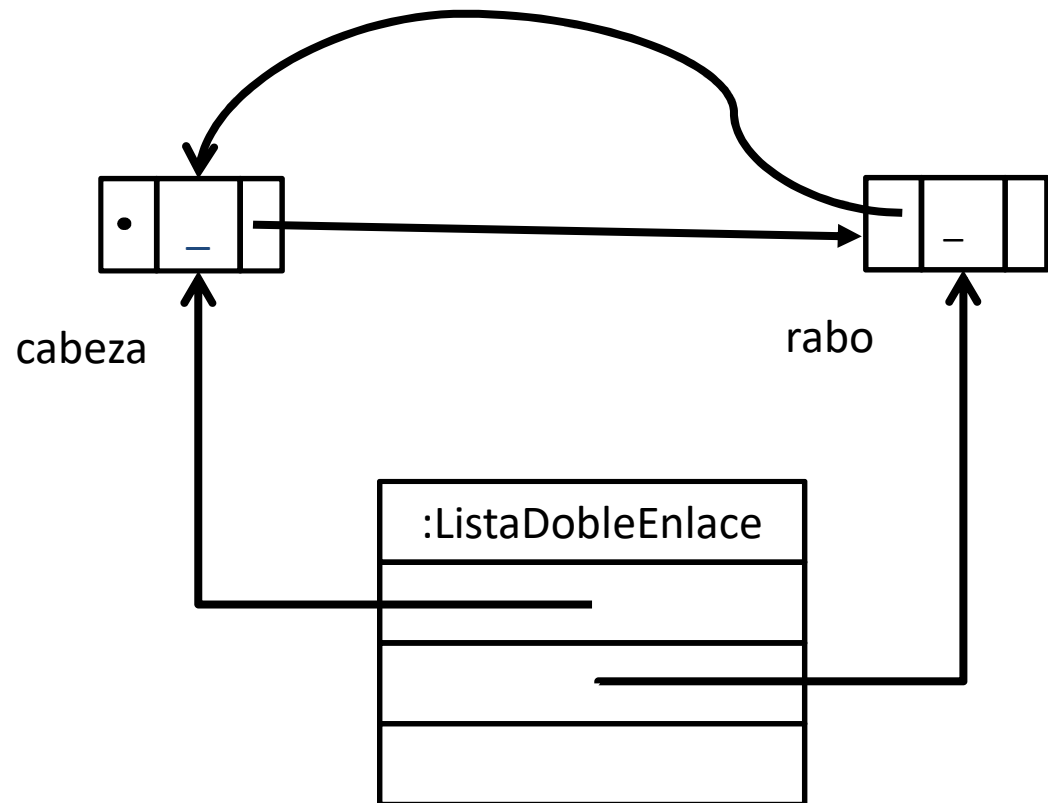
Lista doblemente enlazada: Traza del Constructor

```
public ListaDoblementeEnlazada() {  
    1. cabeza = new DNodeo<E>(null);  
    2. rabo = new DNodeo<E>(null);  
    3. cabeza.setPrev( null );  
    4. cabeza.setNext( rabo );  
}
```



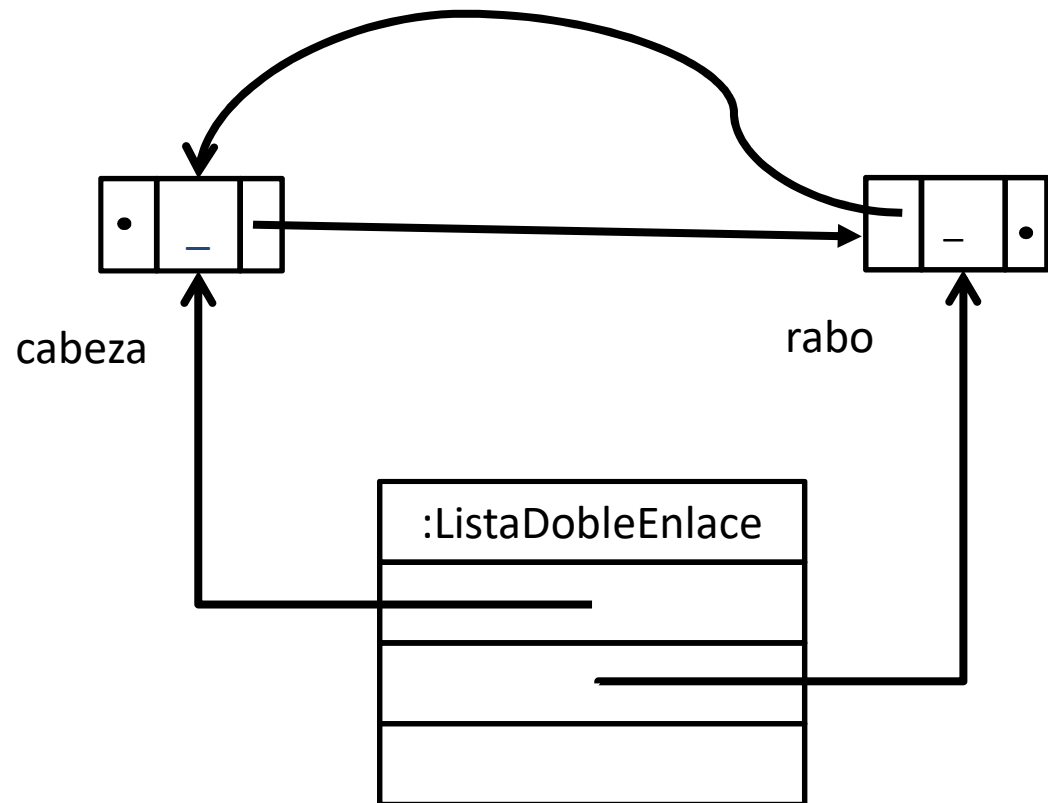
Lista doblemente enlazada: Traza del Constructor

```
public ListaDoblementeEnlazada() {  
    1. cabeza = new DNodeo<E>(null);  
    2. rabo = new DNodeo<E>(null);  
    3. cabeza.setPrev( null );  
    4. cabeza.setNext( rabo );  
    5. rabo.setPrev( cabeza );  
}
```



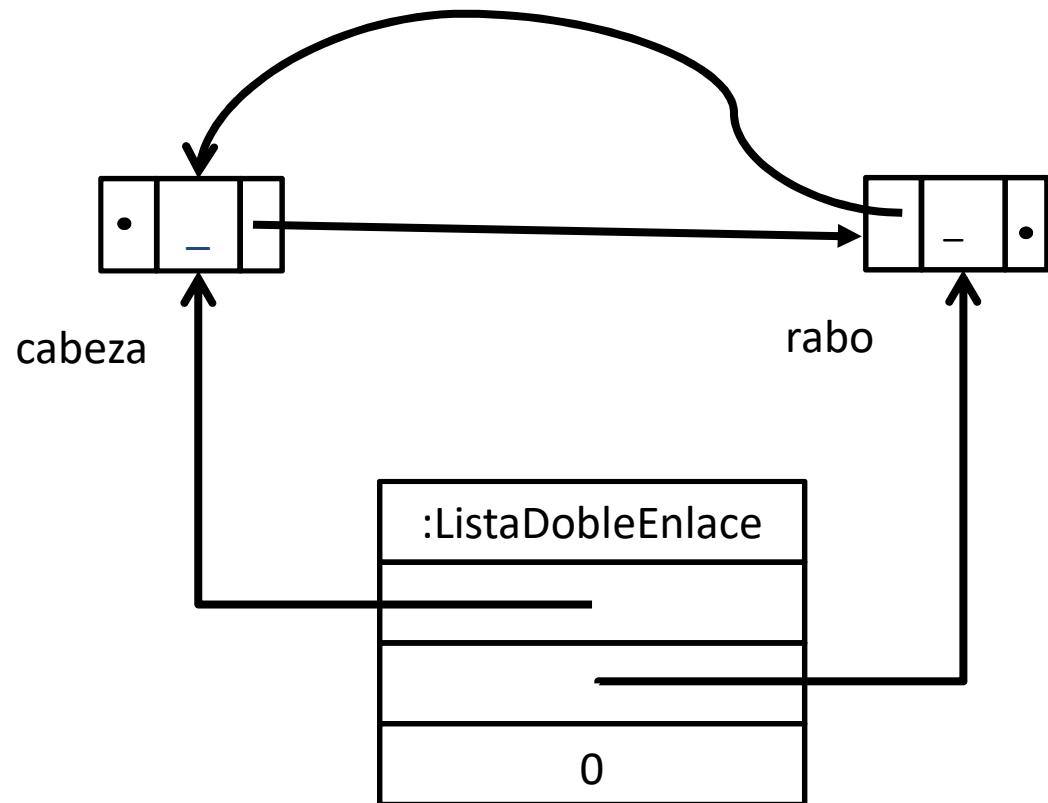
Lista doblemente enlazada: Trazo del Constructor

```
public ListaDoblementeEnlazada() {  
    1. cabeza = new DNodeo<E>(null);  
    2. rabo = new DNodeo<E>(null);  
    3. cabeza.setPrev( null );  
    4. cabeza.setNext( rabo );  
    5. rabo.setPrev( cabeza );  
    6. rabo.setNext( null );  
}
```



Lista doblemente enlazada: Trazado del Constructor

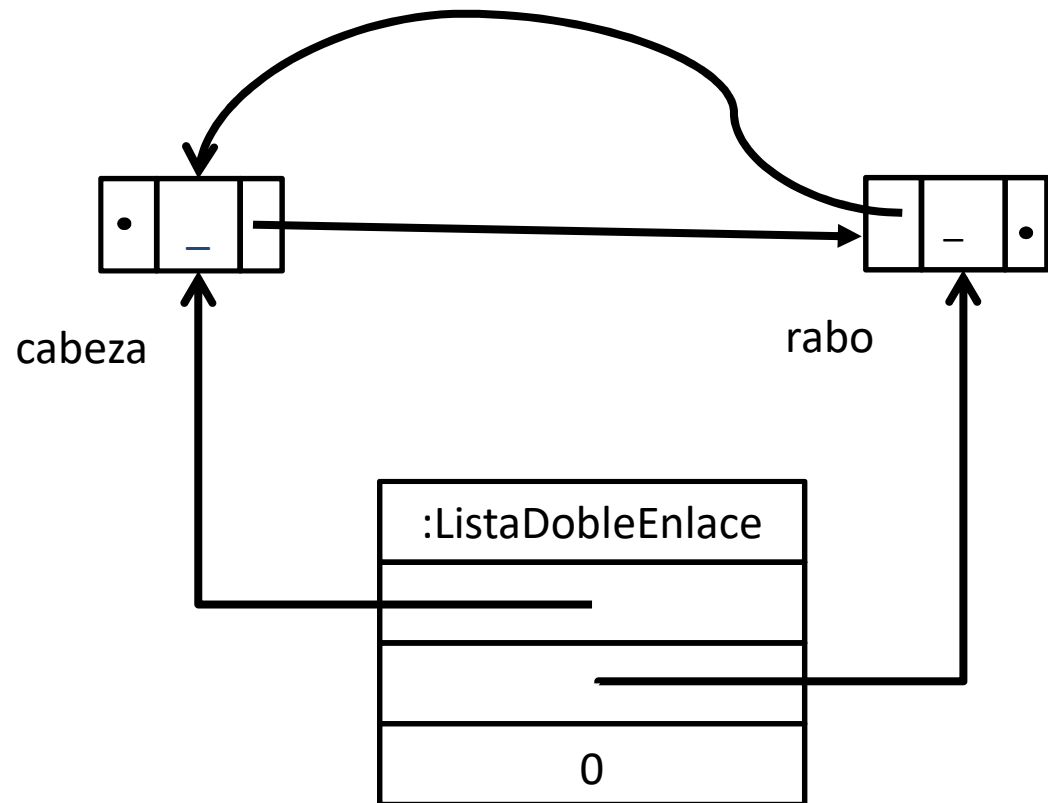
```
public ListaDoblementeEnlazada() {  
    1. cabeza = new DNodeo<E>(null);  
    2. rabo = new DNodeo<E>(null);  
    3. cabeza.setPrev( null );  
    4. cabeza.setNext( rabo );  
    5. rabo.setPrev( cabeza );  
    6. rabo.setNext( null );  
    7. tamaño = 0;  
}
```



Lista doblemente enlazada: Constructor revisado

Si asumimos que el constructor de DNode setea todos los campos en null, se puede hacer un constructor más minimalista:

```
public ListaDoblementeEnlazada() {  
    cabeza = new DNode<E>();  
    rabo = new DNode<E>();  
    cabeza.setNext( rabo );  
    rabo.setPrev( cabeza );  
    tamaño = 0;  
}
```



Lista doblemente enlazada:

Ejemplo de operación

Las operaciones de inserción no tienen casos especiales:

```
public void addAfter(Position<E> p, E element) {  
    DNode<E> pos = checkPosition(p);  
    DNode<E> nuevo = new DNode<E>(element);  
    nuevo.setNext(pos.getNext());  
    nuevo.setPrev(pos);  
    nuevo.getNext().setPrev(nuevo);  
    pos.setNext(nuevo);  
    size++;  
}
```

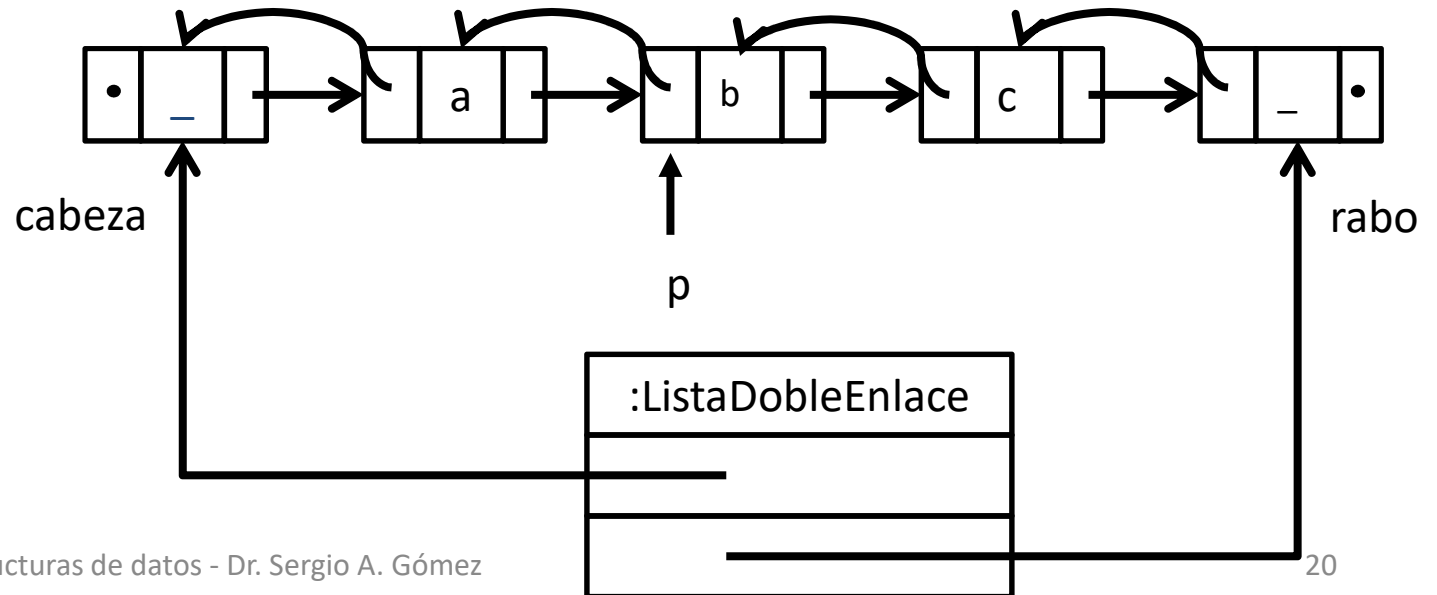
Nota: Ver que es correcta incluso cuando $p == \text{first}()$ y cuando $p == \text{last}()$.

Nota: $\text{addBefore}(p,e)$ y $\text{remove}(p)$ se resuelven con una técnica similar

Lista doblemente enlazada: traza

```
public void addAfter(Position<E> p, E element) {
```

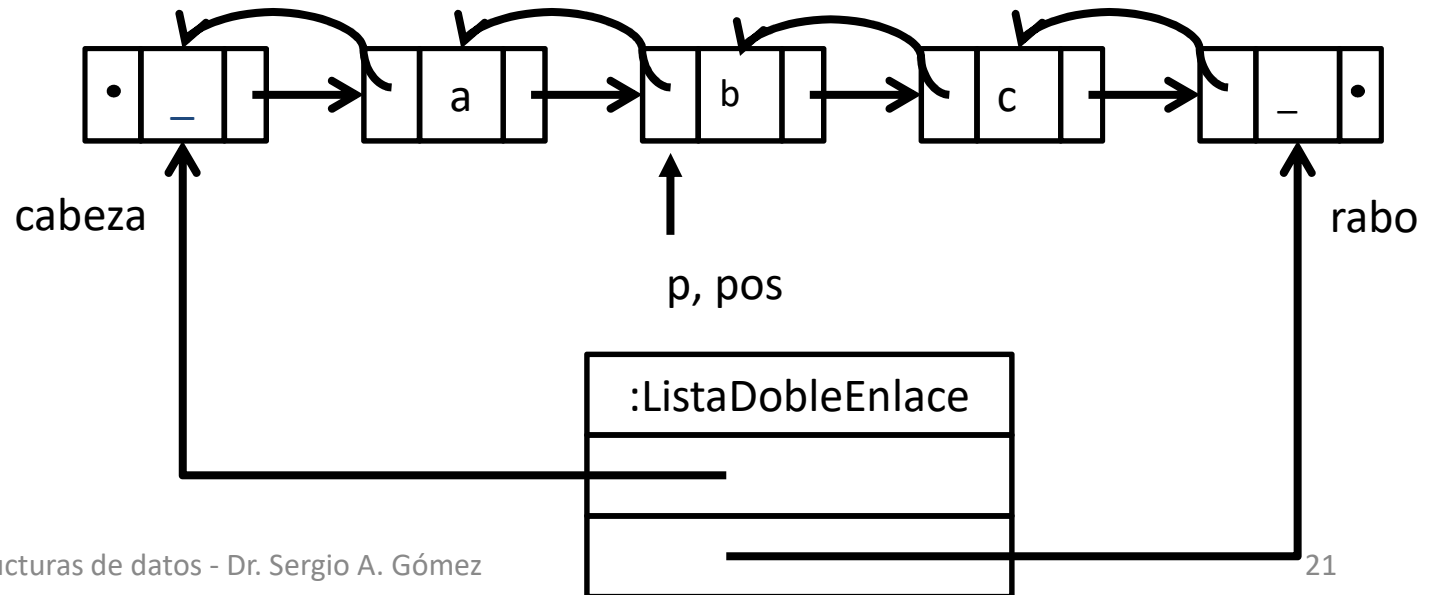
```
}
```



Lista doblemente enlazada: traza

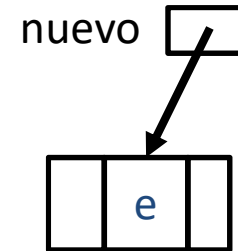
```
public void addAfter(Position<E> p, E element) {  
    DNode<E> pos = checkPosition(p);
```

```
}
```

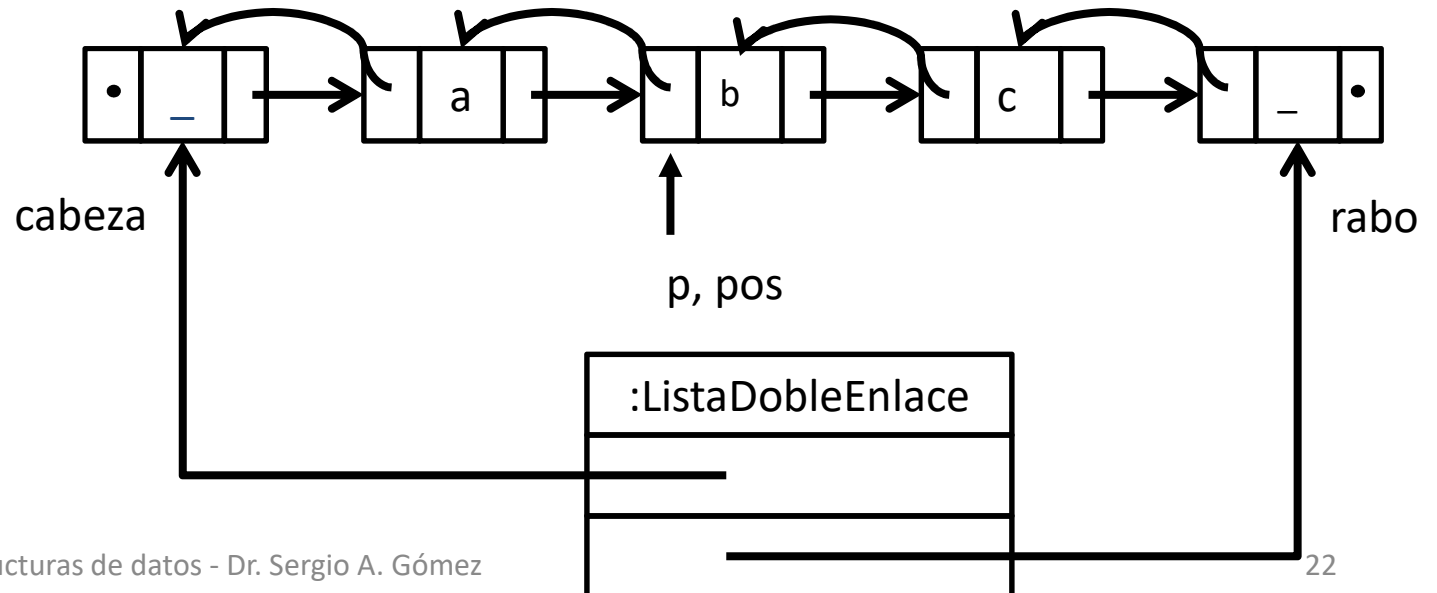


Lista doblemente enlazada: traza

```
public void addAfter(Position<E> p, E element) {  
    DNode<E> pos = checkPosition(p);  
    DNode<E> nuevo = new DNode<E>(element);
```

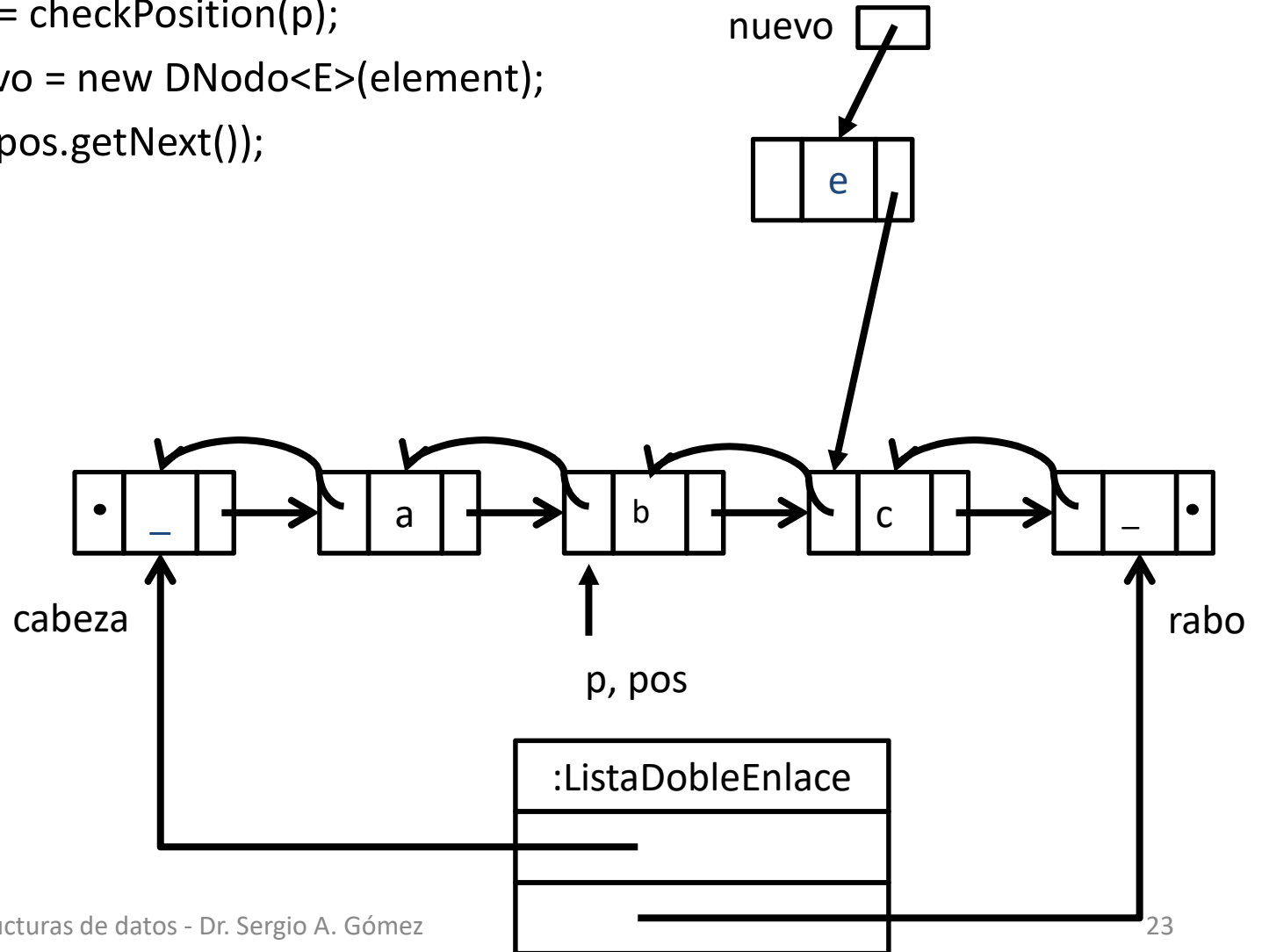


```
}
```



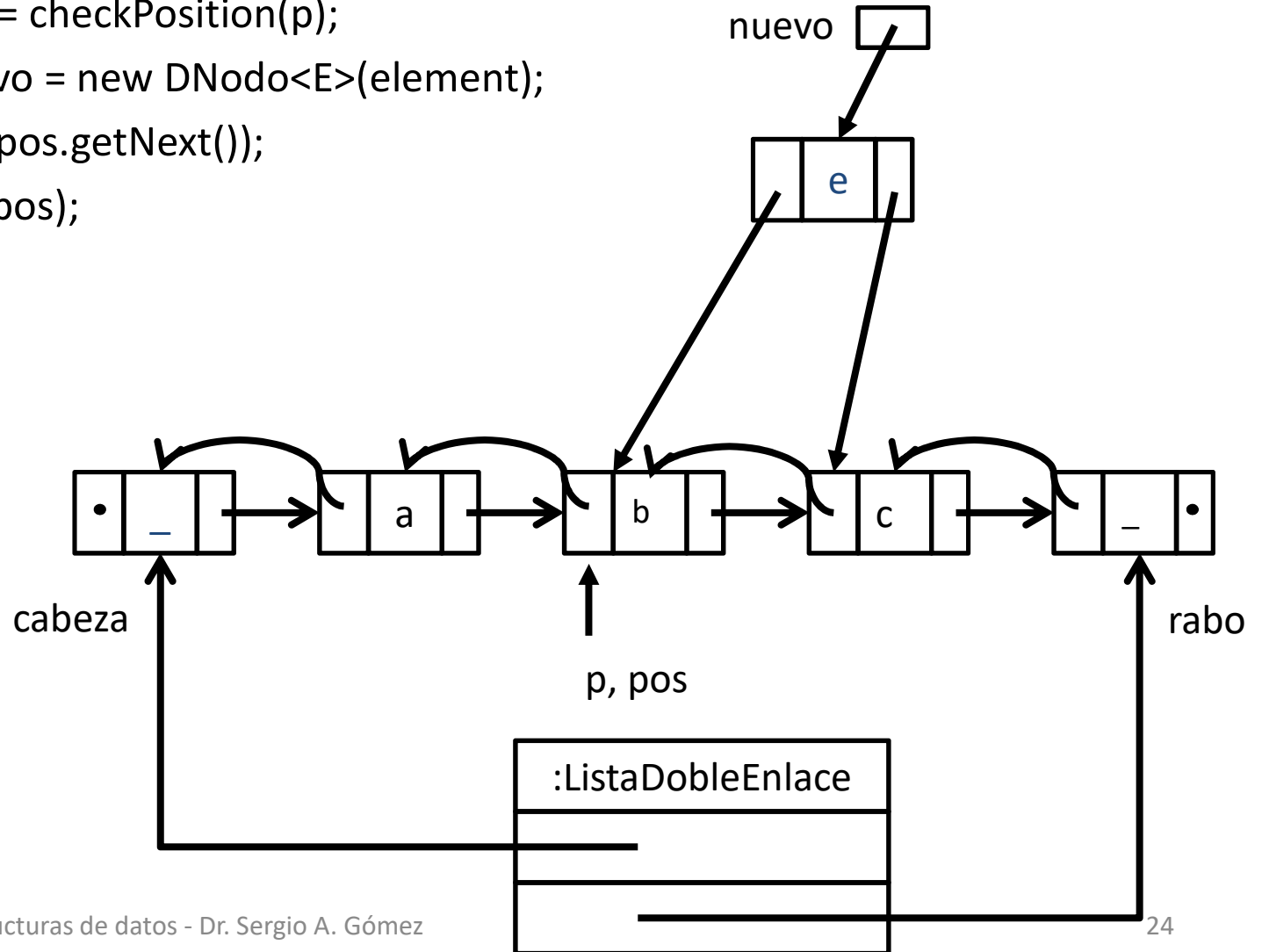
Lista doblemente enlazada: traza

```
public void addAfter(Position<E> p, E element) {  
    DNode<E> pos = checkPosition(p);  
    DNode<E> nuevo = new DNode<E>(element);  
    nuevo.setNext(pos.getNext());  
}
```



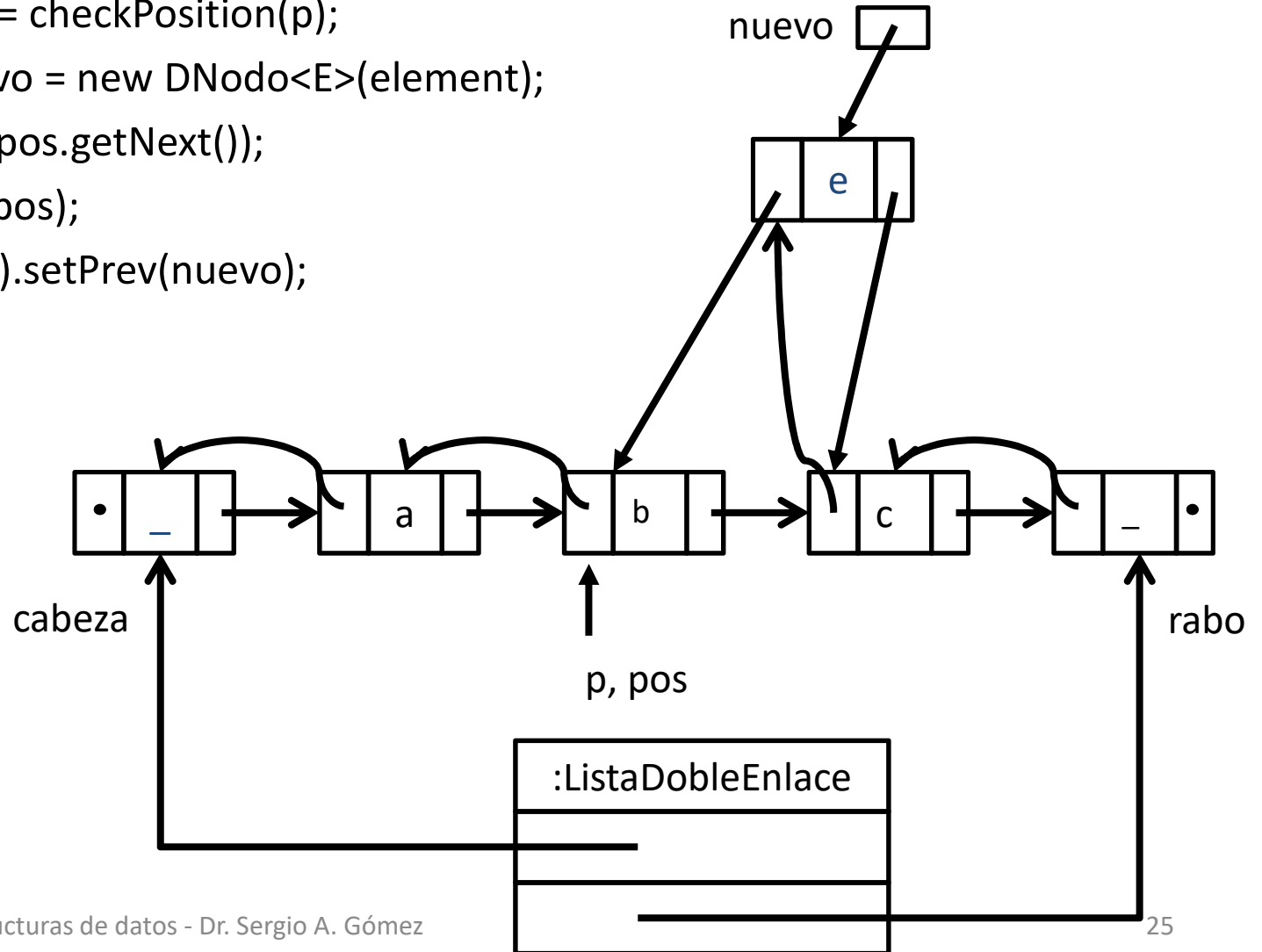
Lista doblemente enlazada: traza

```
public void addAfter(Position<E> p, E element) {  
    DNode<E> pos = checkPosition(p);  
    DNode<E> nuevo = new DNode<E>(element);  
    nuevo.setNext(pos.getNext());  
    nuevo.setPrev(pos);  
}
```



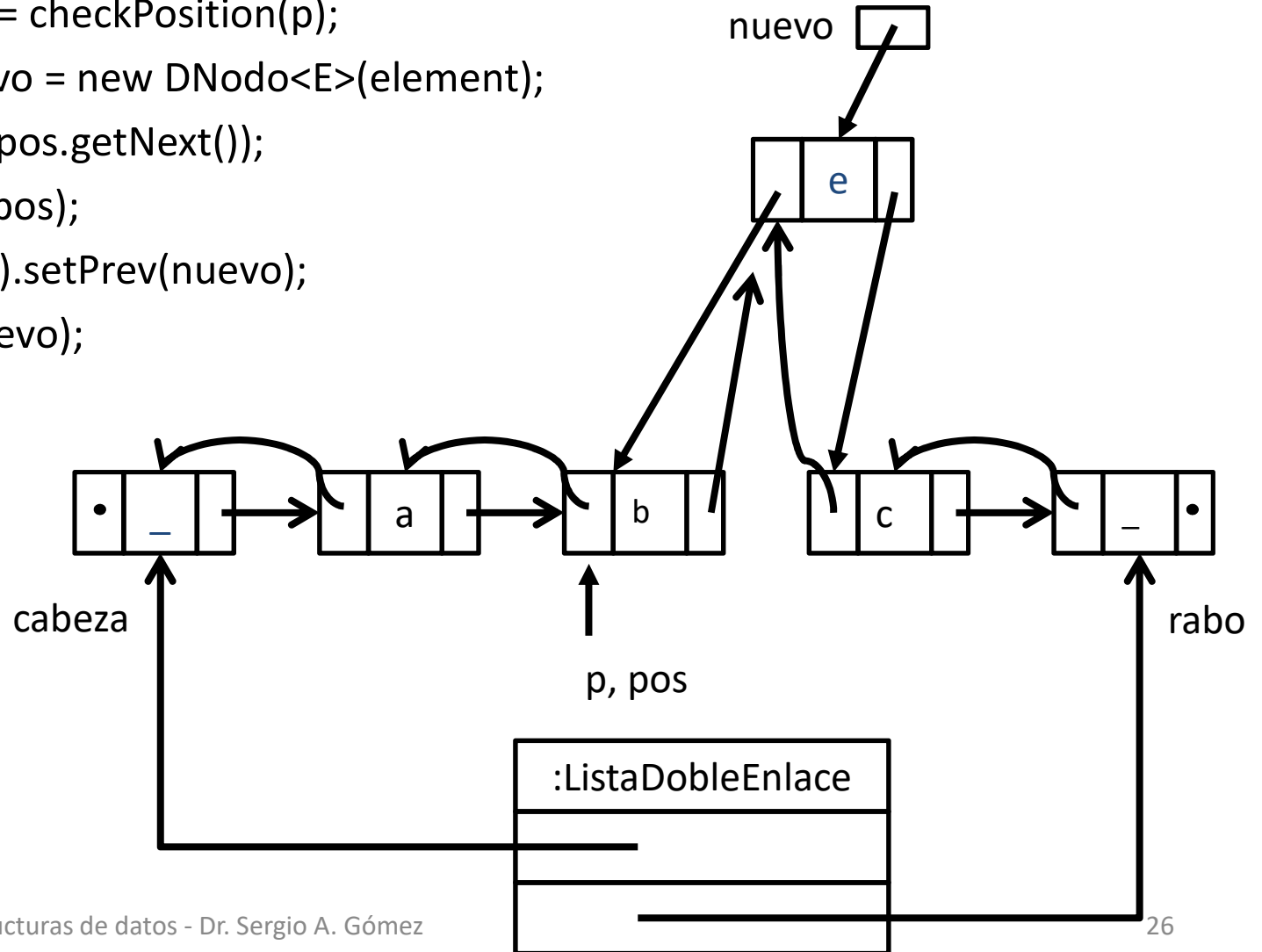
Lista doblemente enlazada: traza

```
public void addAfter(Position<E> p, E element) {  
    DNode<E> pos = checkPosition(p);  
    DNode<E> nuevo = new DNode<E>(element);  
    nuevo.setNext(pos.getNext());  
    nuevo.setPrev(pos);  
    nuevo.getNext().setPrev(nuevo);  
}
```



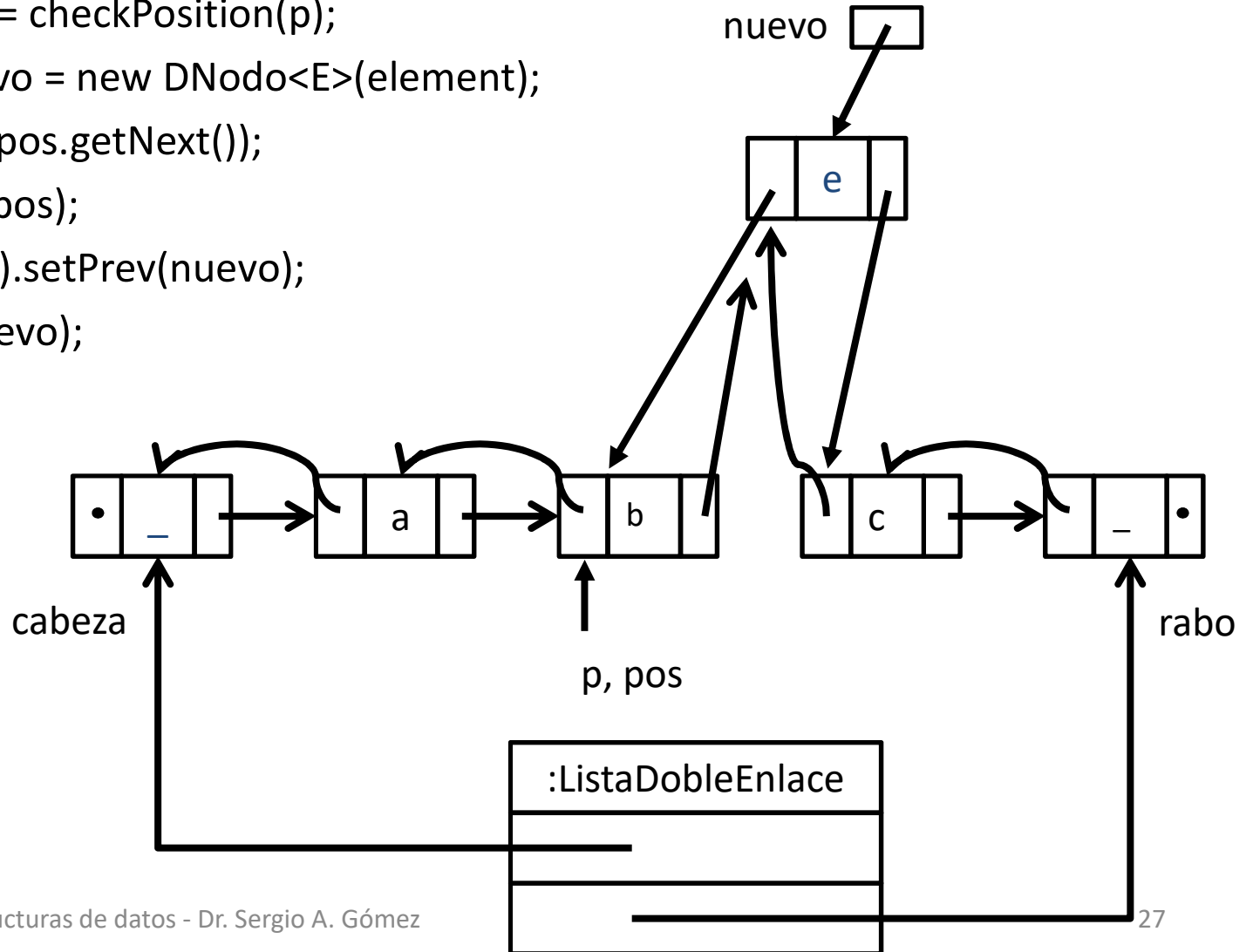
Lista doblemente enlazada: traza

```
public void addAfter(Position<E> p, E element) {  
    DNode<E> pos = checkPosition(p);  
    DNode<E> nuevo = new DNode<E>(element);  
    nuevo.setNext(pos.getNext());  
    nuevo.setPrev(pos);  
    nuevo.getNext().setPrev(nuevo);  
    pos.setNext(nuevo);  
}
```

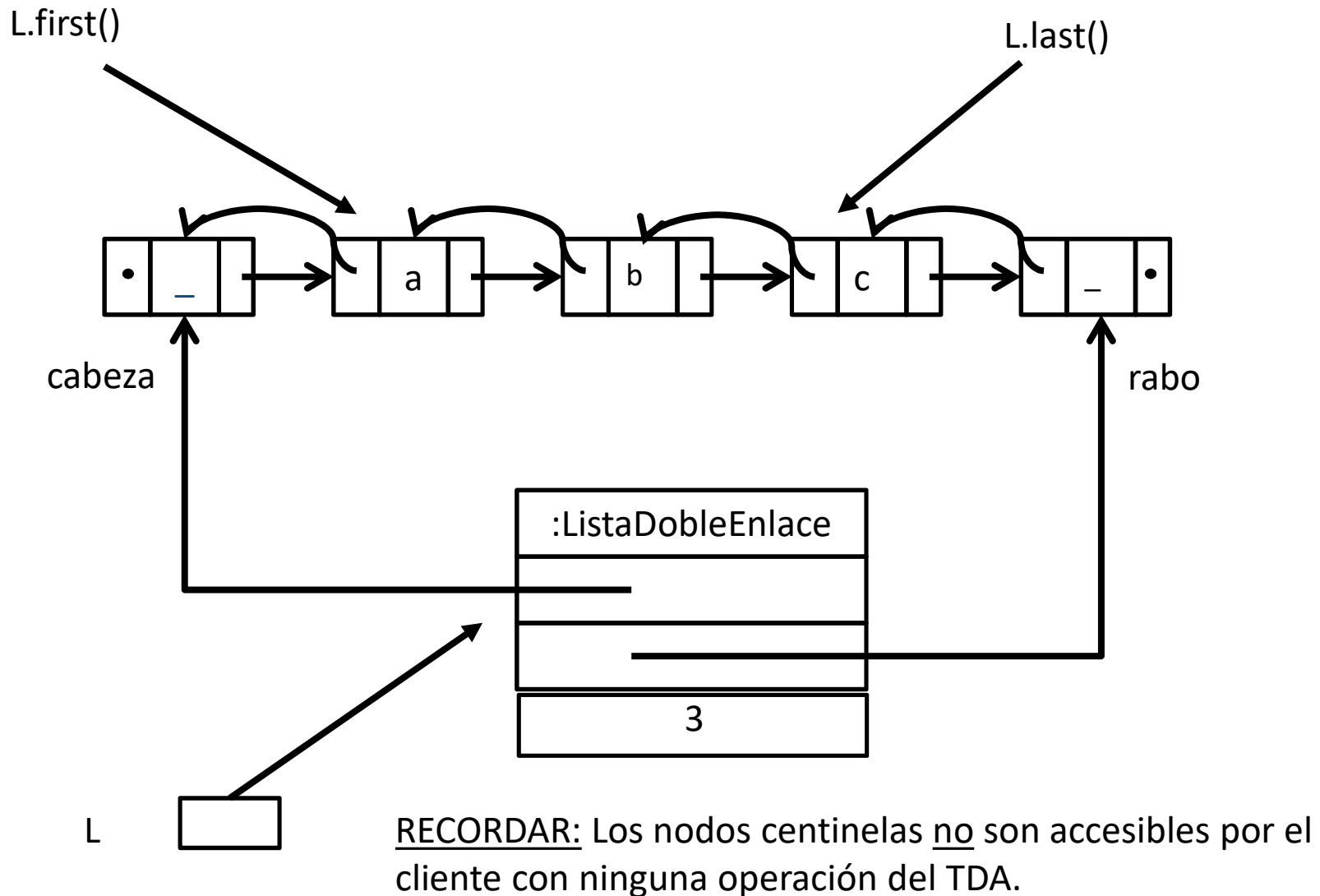


Lista doblemente enlazada: traza

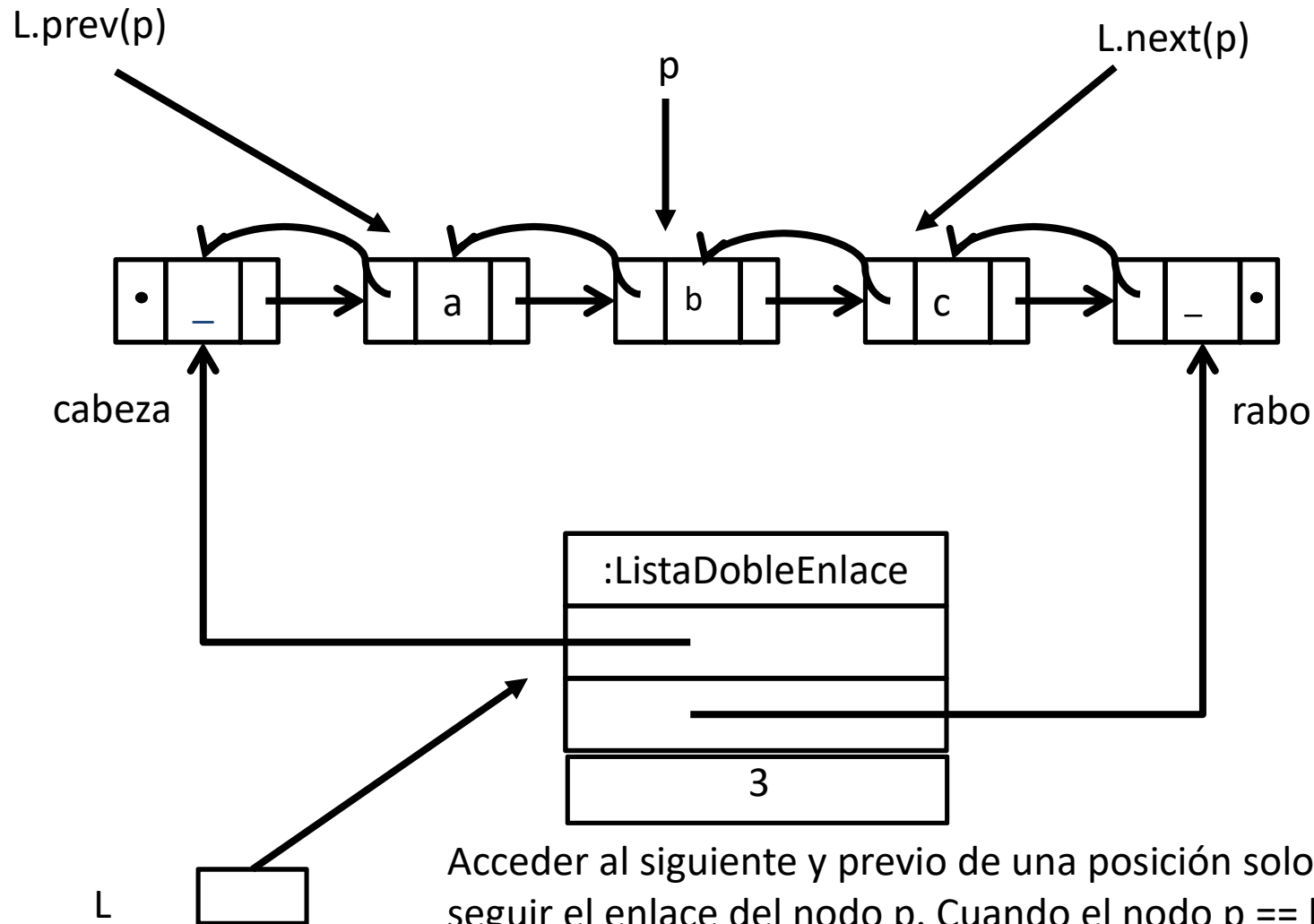
```
public void addAfter(Position<E> p, E element) {  
    DNode<E> pos = checkPosition(p);  
    DNode<E> nuevo = new DNode<E>(element);  
    nuevo.setNext(pos.getNext());  
    nuevo.setPrev(pos);  
    nuevo.getNext().setPrev(nuevo);  
    pos.setNext(nuevo);  
    tamaño++;  
}
```



Lista doblemente enlazada: first y last



Lista doblemente enlazada: next y prev



Acceder al siguiente y previo de una posición solo requiere seguir el enlace del nodo p . Cuando el nodo $p == \text{last}()$, no se puede ejecutar $\text{next}(p)$ y cuando el nodo $p == \text{first}()$, no se puede ejecutar $\text{prev}(p)$

Iteradores

- Un iterador es un patrón de diseño que abstrae el recorrido de los elementos de una colección
- Un iterador consiste de una secuencia S , un elemento corriente y una manera de avanzar al siguiente elemento de S haciéndolo el nuevo elemento corriente
- ADT Iterador (provisto por la interfaz `java.util.Iterator`):
 - `hasNext()`: Testea si hay elementos para seguir iterando
 - `next()`: Retorna el siguiente elemento
- ADT Iterable: Para poder ser iterable una colección debe brindar el método:
 - `iterator()`: Retorna un iterador para los elementos de la colección

```
import java.util.Iterator;
....
Iterator<E> it = colección.iterator();
while( it.hasNext() ) {
    E elem = it.next()
    ... procesar elem ...
}
```

La variable *colección* debe ser de un tipo que implementa la interfaz *java.lang.Iterable<E>*.

Si *colección* = $[a, b, c, d, e]$, *elem* irá tomando los valores a, b, c, d, e a medida que itera el while.

Bucle for-each de Java

La sentencia for-each permite expresar un recorrido de una colección en alto nivel. Supongamos que “colección” es una expresión de un tipo que implementa la interfaz `java.lang.Iterable<E>`:

```
for( E elem : colección )  
    sentencia(elem);
```

El for-each se lee “para cada elem de colección hacer `sentencia(elem)`” y permite ejecutar `sentencia(elem)` sobre cada elemento elem de colección. Además, el for-each es una abreviatura para este código:

```
Iterator<E> it = colección.iterator();  
while( it.hasNext() ) {  
    E elem = it.next();  
    sentencia(elem);  
}
```

Listas iterables

Para tener una lista genérica de elementos de tipo E que soporte la iteración debemos implementar la interfaz *Iterable*< E > y tener dos métodos adicionales llamados *iterator()* y *positions()*.

```
public interface PositionList<E> extends java.lang.Iterable<E> {  
    // Todos los otros métodos que ya estudiamos  
    ....  
    // Más dos nuevos:  
  
    // Retorna un iterador para los elementos de la lista:  
    // Esta operación es requerida por java.lang.Iterable  
    public Iterator<E> iterator();  
  
    // Devuelve una colección iterable de posiciones:  
    // Esta operación es propuesta por Goodrich y Tamassia.  
    public Iterable<Position<E>> positions();  
}
```


Listas iterables: Ejemplos

Hallar el máximo elemento de una lista de enteros positivos.

```
static int hallarMaximo( PositionList<Integer> lista )
{
    int maximo = 0;
    Iterator<Integer> it = lista.iterator();
    while ( it.hasNext() ) {
        Integer elem = it.next();
        if( elem > maximo )
            maximo = elem;
    }
    return maximo;
}
```

Listas iterables: Ejemplos

Hallar el máximo elemento de una lista de enteros positivos.
Ahora lo resolvemos usando un for-each:

```
static int hallarMaximo( PositionList<Integer> lista )  
{  
    int maximo = 0;  
    for( Integer elem : lista )  
        if( elem > maximo )  
            maximo = elem;  
    return maximo;  
}
```

Listas iterables: Ejemplos

Problema: Buscar un elemento x en una lista l (recorrido no exhaustivo).

Solución estructurada:

```
public static <E> boolean buscar(PositionList<E> l, E x) {  
    Iterator<E> it = l.iterator();  
    boolean encuentre = false;  
    while( it.hasNext() && !encuentre )  
        if( it.next().equals(x) )  
            encuentre = true;  
    return encuentre;  
}
```

Discusión:

La solución no estructurada es más minimalista y la solución estructurada es más verborrágica.

Solución no estructurada:

```
public static <E> boolean buscar(PositionList<E> l, E x) {  
    for( E e : l )  
        if( e.equals(x) ) return true;  
    return false;  
}
```

Listas iterables: Ejemplos

Problema: Programar el método toString() para la lista iterable.

Usa el iterador de PositionList para devolver un string con el formato $[x_1, x_2, \dots, x_n]$ a partir de la lista receptora en $O(n)$.

```
import java.lang.Iterator;
```

```
public String toString() {  
    Iterator<E> it = iterator();           // Le pido el iterador a la lista this.  
    String s = "[";  
    while( it.hasNext() ) {  
        s += it.next();                   // Hay un cast implícito de E a String, equivale a: s+=it.next().toString();  
        if( it.hasNext() )                // Hago append de una coma si quedan elementos  
            s += ", ";  
    }  
    s += "];"  
    return s;  
}
```

Listas iterables: Ejemplos

Problema: Eliminar la primera aparición de x en una lista l.

Solución estructurada:

```
public static <E> void eliminar( PositionList<E> l, E x ) {  
    Iterable<Position<E>> lpos = l.positions();  
    boolean encuentre = false;  
    Iterator<Position<E>> it = lpos.iterator();  
    while( it.hasNext() && !encontre ) {  
        Position<E> p = it.next();  
        if (p.element().equals(x)) {  
            encuentre = true;  
            l.remove(p);  
        }  
    }  
}
```

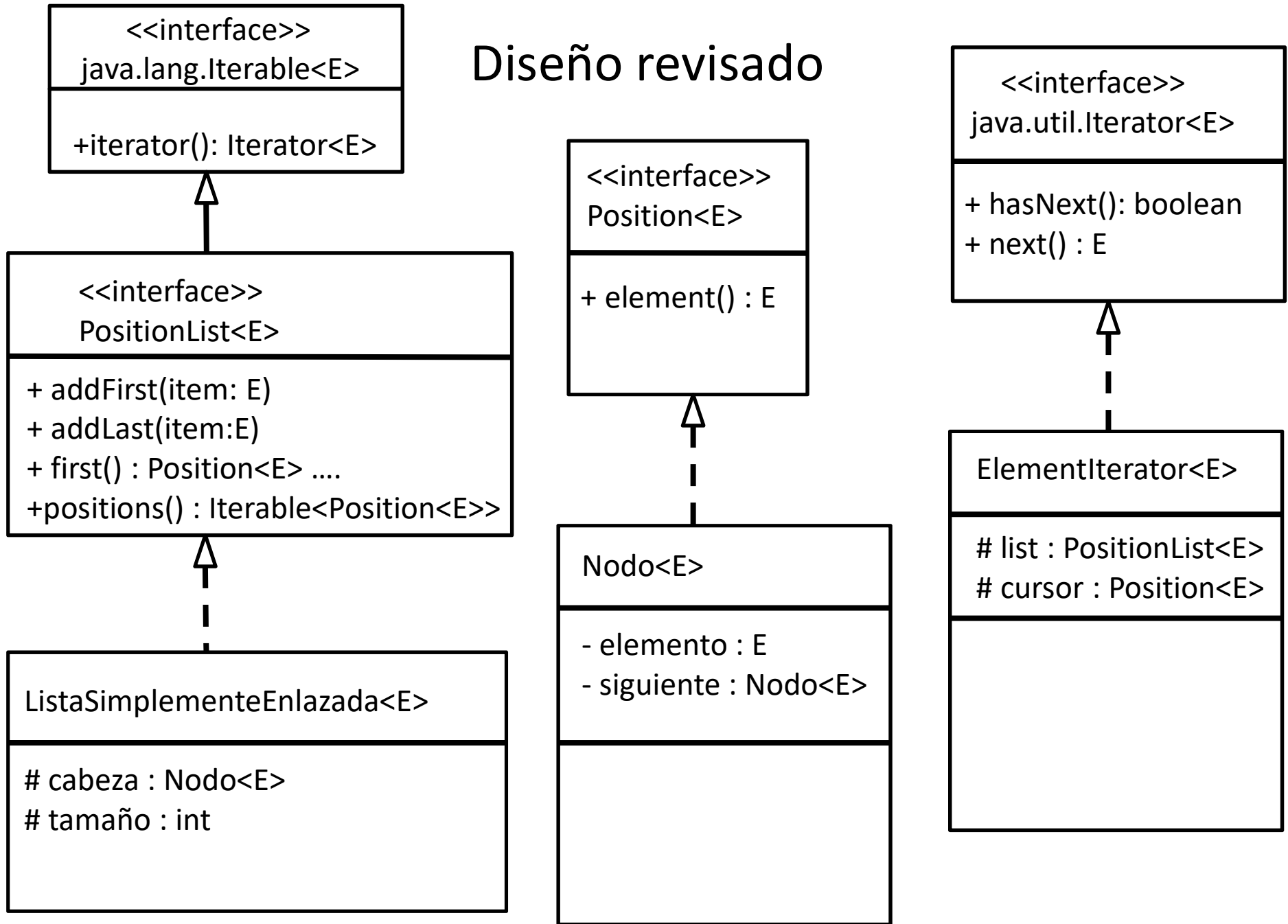
Listas iterables: Ejemplos

Problema: Eliminar la primera aparición de x en una lista l.

Solución no estructurada:

```
public static <E> void eliminar( PositionList<E> l, E x ) {  
    for( Position<E> p : l.positions() )  
        if ( p.element().equals(x) ) {  
            l.remove(p);  
            return;  
        }  
}
```

Diseño revisado



Método iterator de la clase ListaSimplementeEnlazada

Delegaremos en una clase ElementIterator que programaremos luego:

```
import java.util.Iterator;

public Iterator<E> iterator(){
    // Creo un elementIterator sobre la lista this a iterar
    return new ElementIterator<E>( this );
}
```


Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

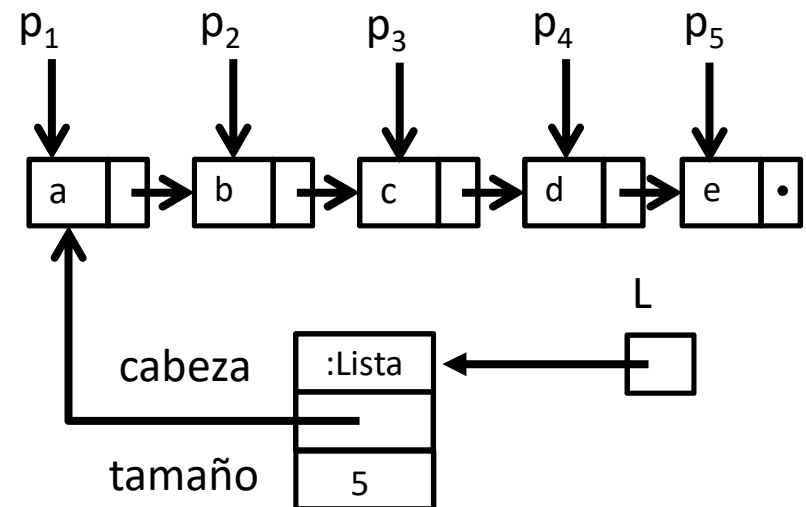
```
Iterador<E> it = L.iterator();
```

```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

```
    ...
```

```
}
```



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

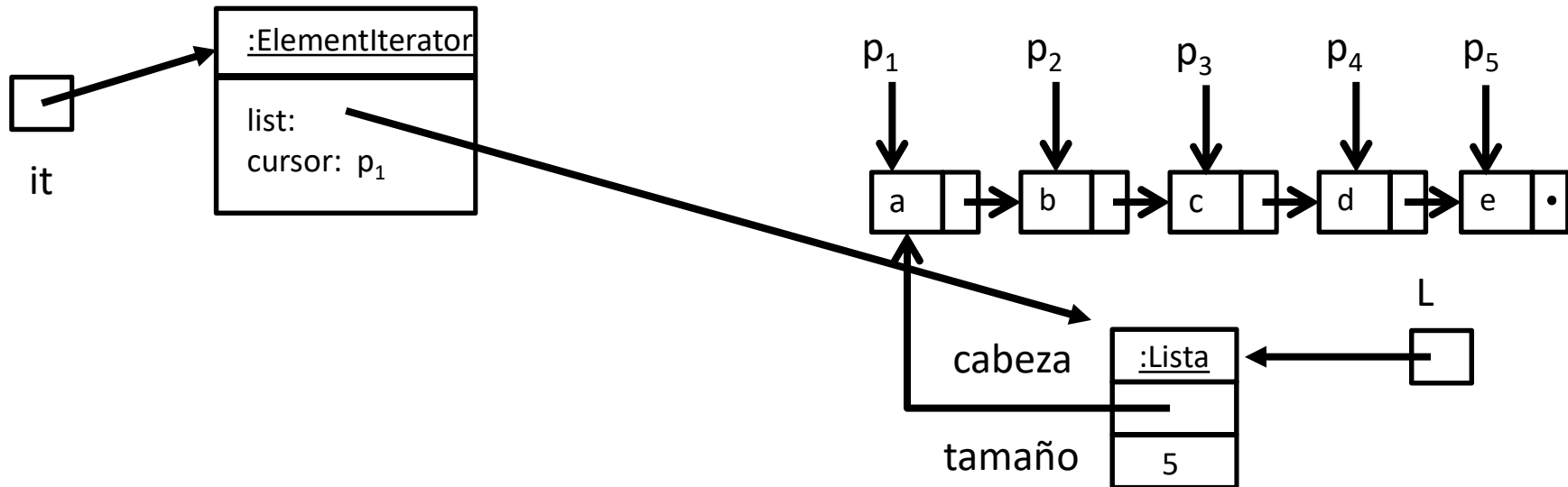
```
Iterador<E> it = L.iterator();
```

```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

```
    ...
```

```
}
```



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

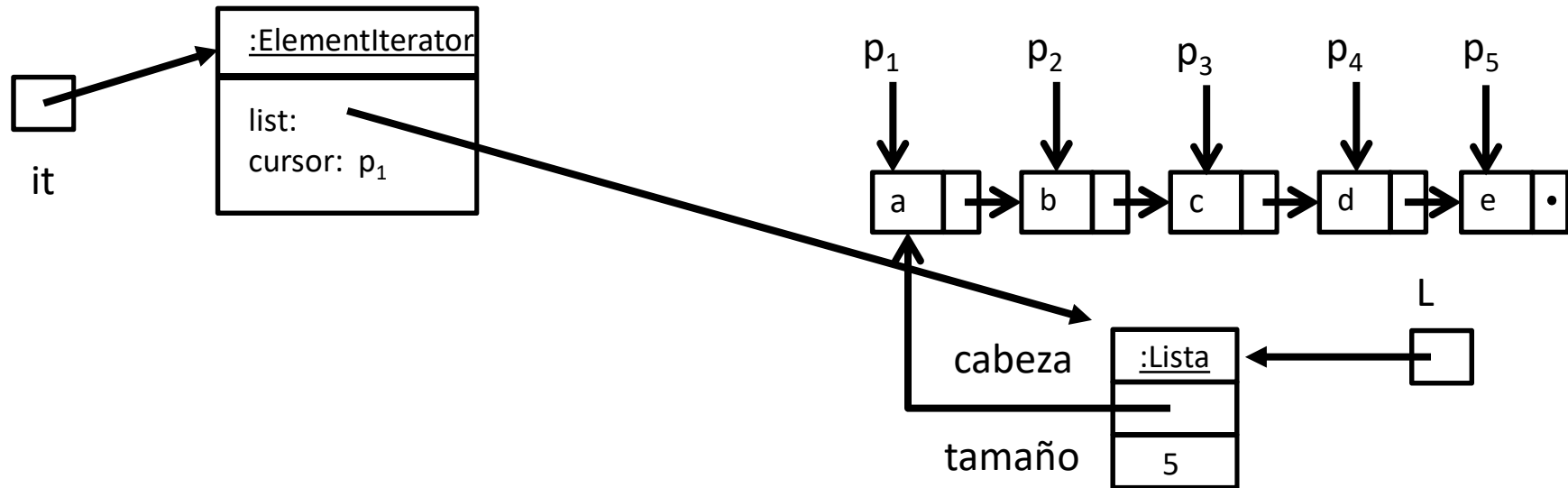
```
Iterador<E> it = L.iterator();
```

```
while (it.hasNext() ) {   hasNext retorna true porque cursor = p1 != null
```

```
    E x = it.next();
```

```
    ...
```

```
}
```



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

```
Iterador<E> it = L.iterator();
```

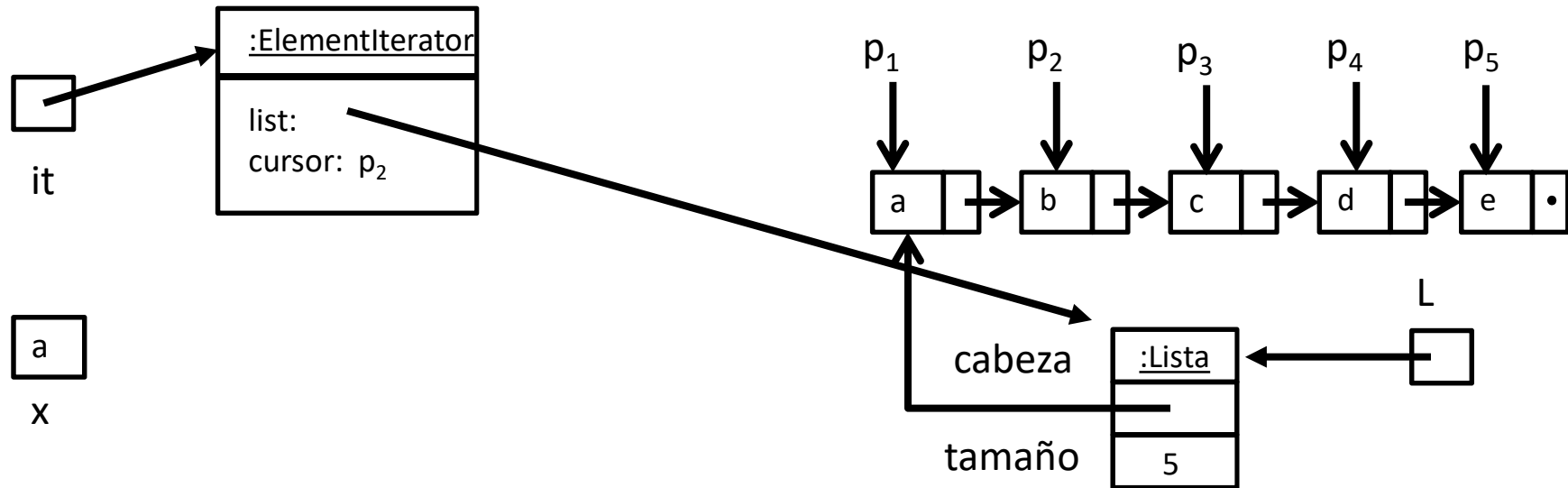
```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

next retorna a y avanza el cursor a p_2

```
    ...
```

```
}
```



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

```
Iterador<E> it = L.iterator();
```

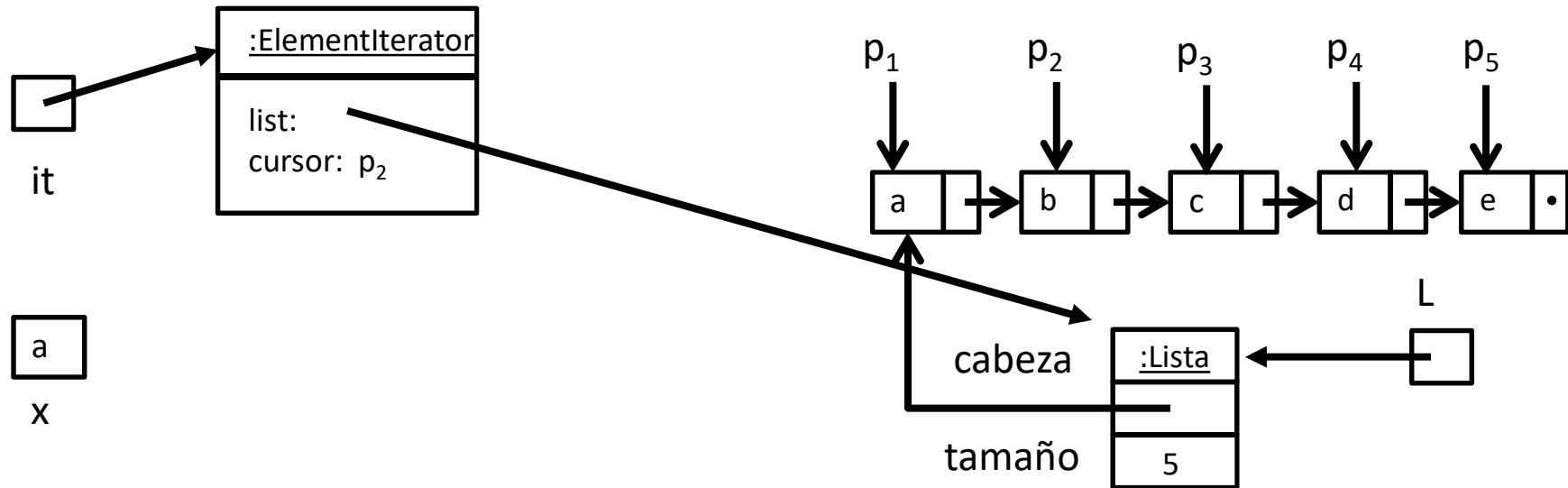
```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

```
    ...
```

```
}
```

hasNext retorna true porque $\text{cursor} = p_2 \neq \text{null}$



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

```
Iterador<E> it = L.iterator();
```

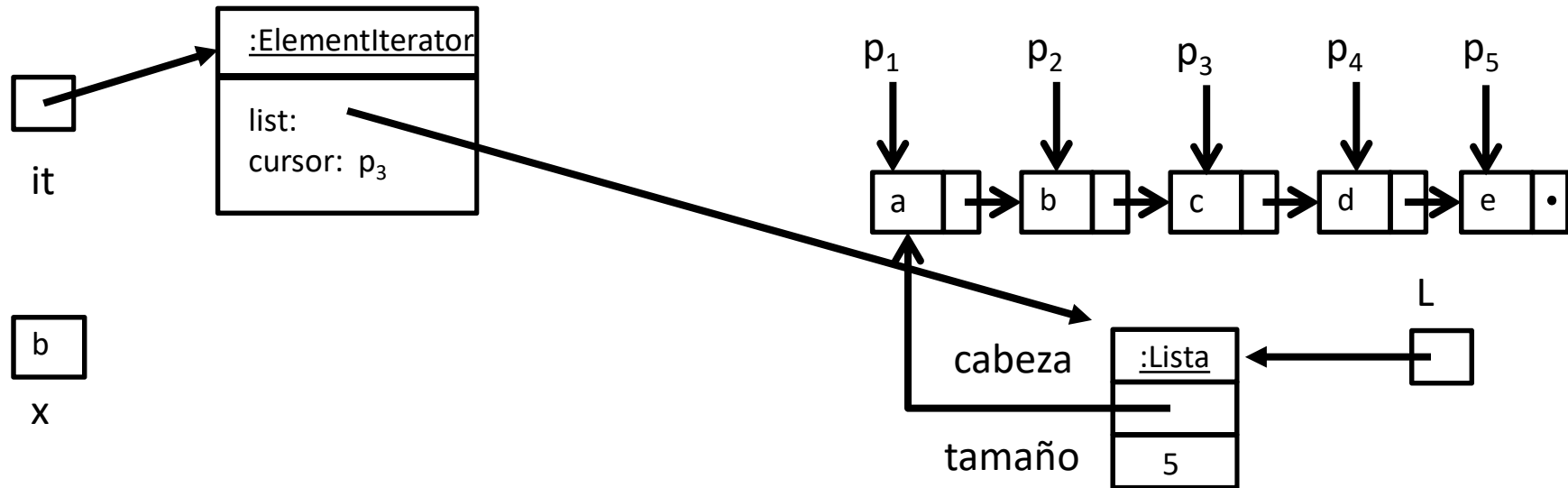
```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

next retorna b y avanza el cursor a p_3

```
    ...
```

```
}
```



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

```
Iterador<E> it = L.iterator();
```

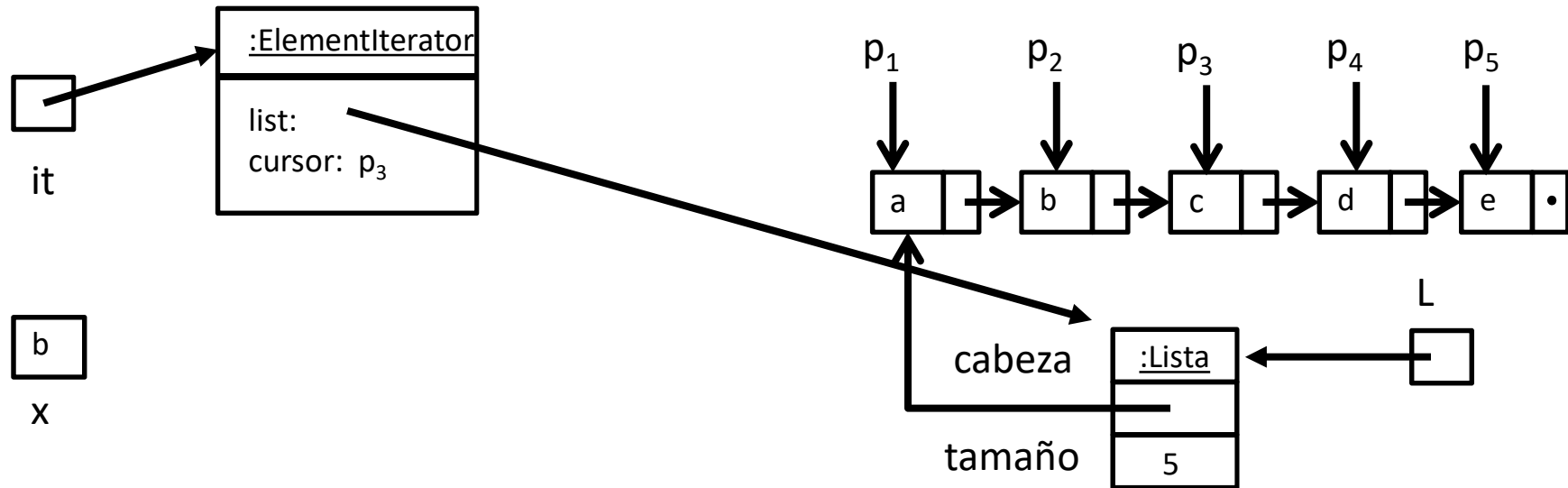
```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

```
    ...
```

```
}
```

hasNext retorna true porque $\text{cursor} = p_3 \neq \text{null}$



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

```
Iterador<E> it = L.iterator();
```

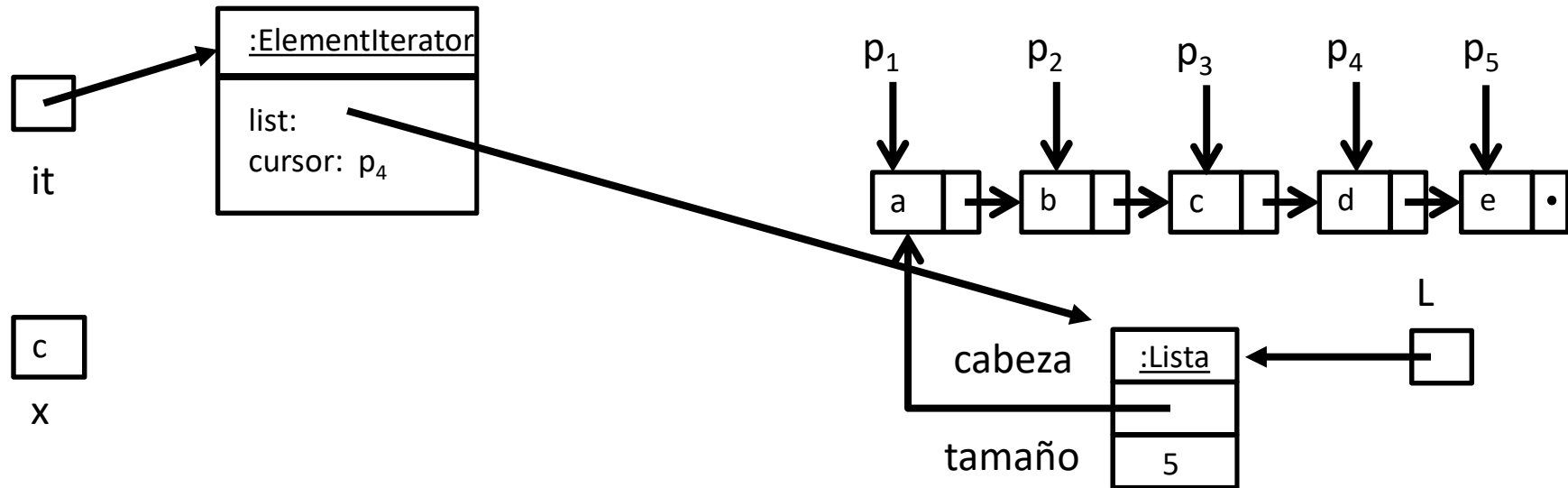
```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

next retorna c y avanza el cursor a p_4

```
    ...
```

```
}
```



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

```
Iterador<E> it = L.iterator();
```

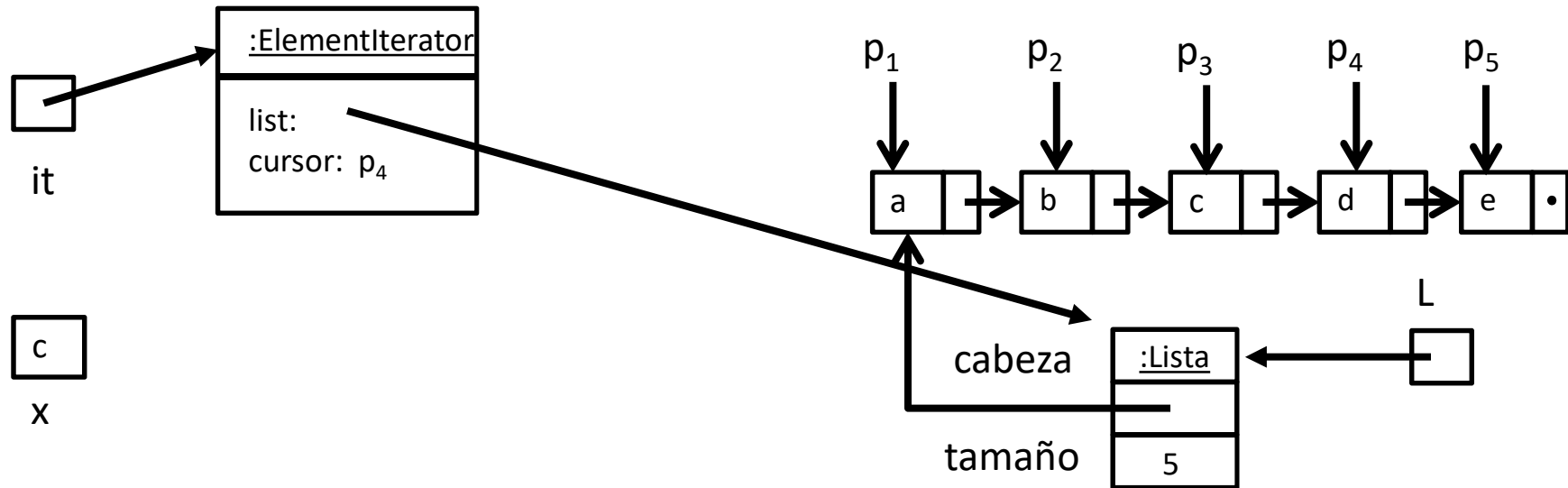
```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

```
    ...
```

```
}
```

hasNext retorna true porque $\text{cursor} = p_4 \neq \text{null}$



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

```
Iterador<E> it = L.iterator();
```

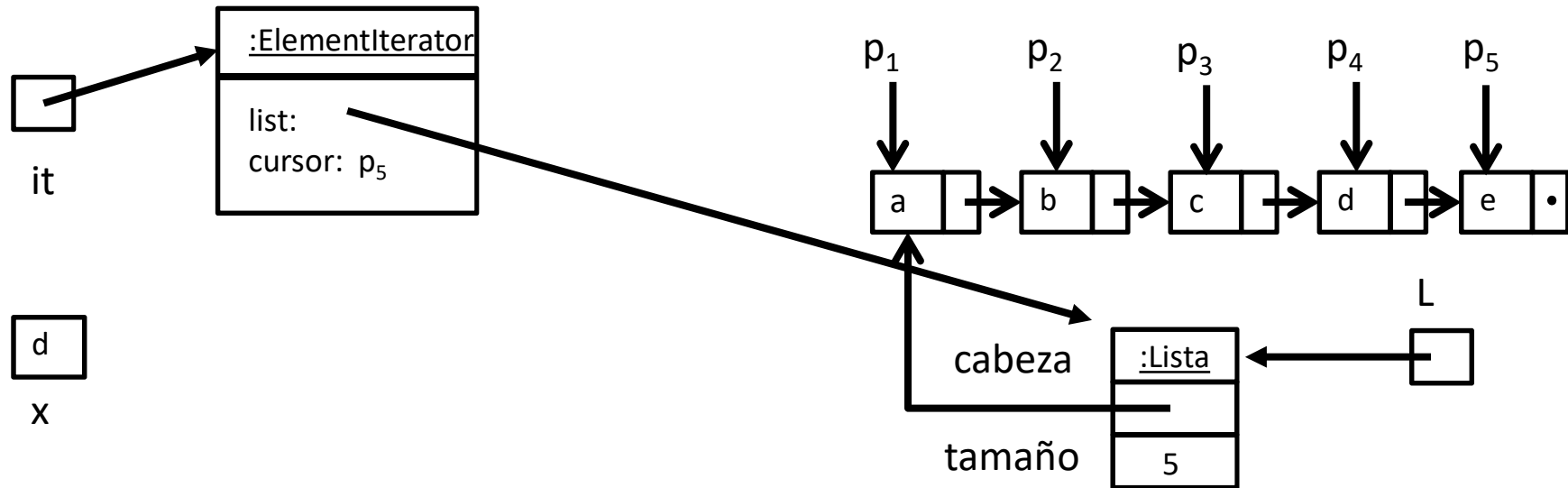
```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

next retorna d y avanza el cursor a p₅

```
    ...
```

```
}
```



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

```
Iterador<E> it = L.iterator();
```

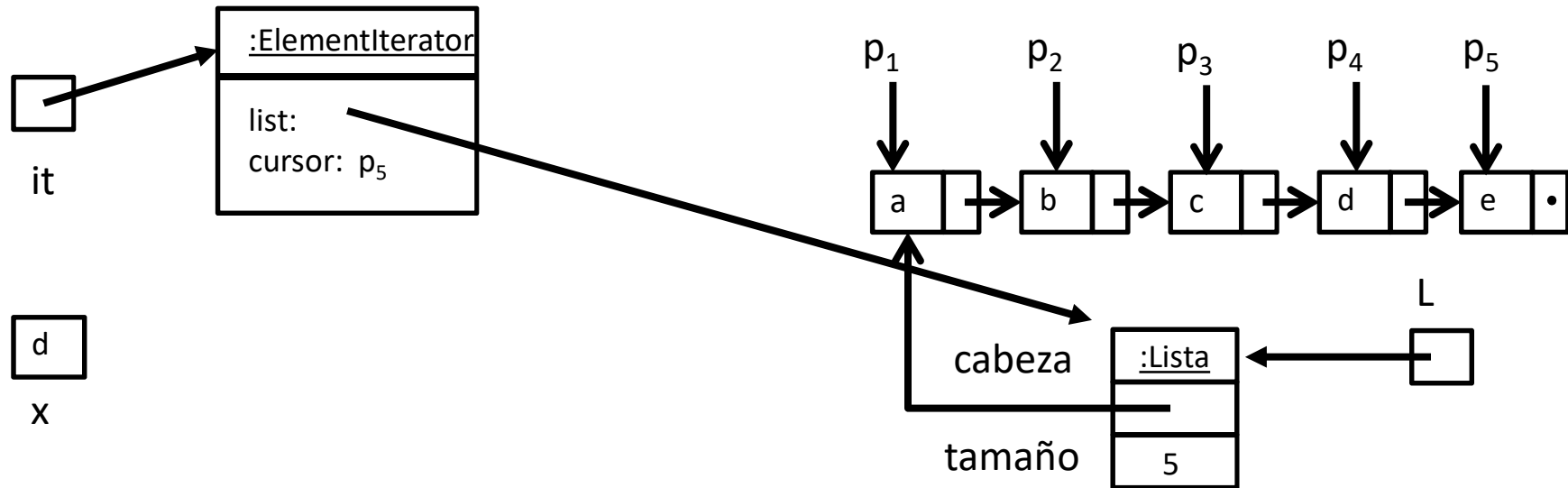
```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

```
    ...
```

```
}
```

hasNext retorna true porque $\text{cursor} = p_5 \neq \text{null}$



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

```
Iterador<E> it = L.iterator();
```

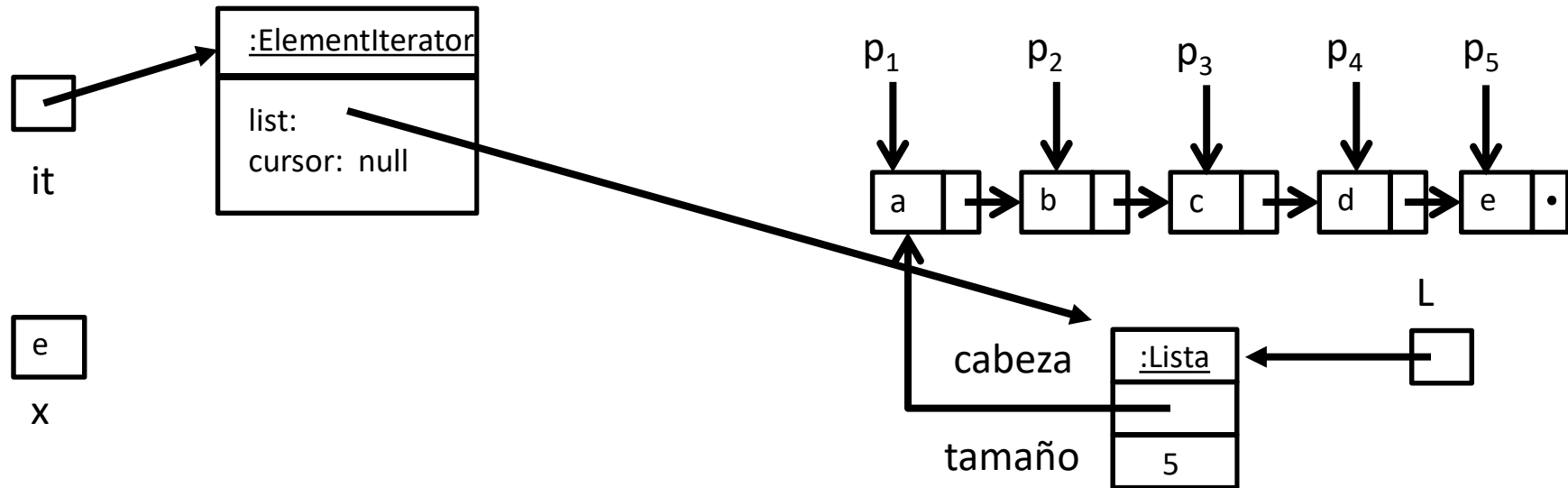
```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

next retorna e y avanza el cursor a null

```
    ...
```

```
}
```



Clase ElementIterator: Intuición

El iterador conoce la lista *list* que itera y la posición corriente llamada *cursor*.

El significado del código siguiente es:

```
Iterador<E> it = L.iterator();
```

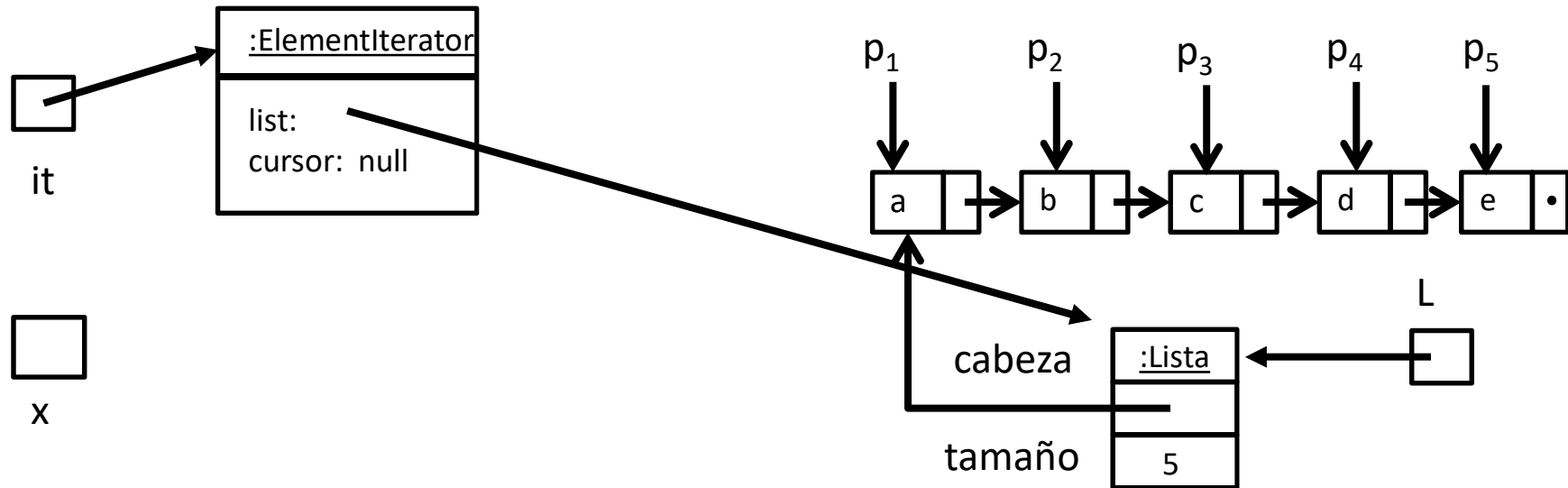
```
while (it.hasNext() ) {
```

```
    E x = it.next();
```

```
    ...
```

```
}
```

hasNext retorna false porque *cursor* = null = null



Clase ElementIterator

```
import java.util.*;

public class ElementIterator<E> implements Iterator<E> {
    protected PositionList<E> lista; // Lista a iterar
    protected Position<E> cursor; // Posición del elemento corriente

    public ElementIterator (PositionList<E> l ) {
        lista = l; // Guardo la lista a iterar
        if (lista.isEmpty()) cursor = null; // Si la lista está vacía, la posición corriente es nula
        else cursor = lista.first(); // Sino la posición corriente es la primera de la lista
    }

    // Devuelve true si hay algún elemento más para ver.
    // Hay siguiente si el cursor no está más allá de la última posición
    public boolean hasNext() { return cursor != null; }

    // Devuelve el siguiente elemento y avanza el cursor. Falla si hasNext es falso.
    public E next () {
        if ( cursor == null ) // Si el cursor es null, el cliente no testeó que hasNext fuera true
            throw new NoSuchElementException ("Iterador de lista: No hay siguiente");
        E resultado = cursor.element(); // Salvo el elemento corriente
        cursor = (cursor == lista.last()) ? null : lista.next(cursor); // Avanzo a la siguiente posición
        return resultado; // Retorno el elemento salvado
    }

    // Remueve el último elemento retornado por el iterador
    public void remove() { /* No lo haremos. */ }
}
```

Patrón adaptador (Adapter)



- El *adaptador* está inspirado en los enchufes adaptadores que permiten usar un enchufe de 2 patas redondas como si fuera uno de 2 patas planas.
- Objetivo: Implementar una pila usando una lista.
- El patrón de diseño Adaptador permite usar una clase para brindar la funcionalidad de otra clase.
- Forma de usarlo:
 - implementar una pila mediante un atributo de tipo array list / PositionList
 - Cada operación de pila se implementa con una operación de array list (es decir, la pila *delega* en array list/PositionList).

Adaptador de pilas

(Implementación de pila con PositionList)

PilaConLista<E>
l : PositionList<E>

Método de la pila	Implementación con PositionList l
size()	l.size()
isEmpty()	l.isEmpty()
push(x)	l.addFirst(x)
pop()	l.remove(l.first())
top()	l.first().element()

Tarea: Implementar una cola usando una PositionList.

Resumen de la clase

1. Listas simplemente enlazadas con enlace al primer y último elemento
 1. `addLast` y `last` se hacen sin recorrido
 2. `prev` y `addBefore` requieren recorrer desde el comienzo ☹️
 3. Hay más casos especiales en las operaciones ☹️
2. Listas doblemente enlazadas con celdas centinelas inicial y final:
 1. Ninguna operación requiere recorrido 😊
 2. No hay casos especiales cuando la lista está vacía (no se ponen ifs) 😊
3. Iteradores: Permiten abstraer el recorrido de la ED 😊
 1. Interfaces *Iterable*<E> e *Iterator*<E> (de Java)
 2. Clase *ElementIterator*<E> (de usuario)
4. Adaptadores: Permiten reusar otras ED 😊
 1. Pilas y colas implementadas con listas (la pila tiene un atributo lista)

Bibliografía

- Goodrich & Tamassia. Data Structures and Algorithms in Java, 4th Edition, capítulo 6.